UNIVERSITY OF THE PEOPLE

59 Seconds: A Projector-Optimized, Region-Localized ESL Speaking Game

A Capstone Project Report of MSIT 5910

Submitted by:

Jacobus Barnard (S542854)

For the partial fulfilment of the requirements for the degree of

Master of Science in Information Technology

Supervised by:

Dr. Sirisha Pavuluri

Department of CS & MSIT

University of the People, Pasadena, CA 91101, United States

Term 5, August 2026

ABSTRACT

Speaking is usually the hardest skill for English as a Second Language (ESL) students to practice, mainly because of speaking anxiety, and most educational apps make it worse by pushing every student onto an individual phone screen with culturally generic content. This capstone designed and built 59 Seconds, a web-based speaking game modeled on the party game 30 Seconds but engineered for a one-to-many classroom: the teacher runs it on a single PC and projector, one student describes the five words on a card, and classmates guess before a 59-second timer runs out. The system is a serverless web application built with React 19 and TanStack Start, hosted on Vercel, with Supabase providing the PostgreSQL database, authentication, and Row-Level Security, and Paddle handling payments as merchant of record. The finished product is live at 59seconds.live with twelve region-localized packs holding 2,880 cards, taking real payments verified end to end including refunds. Evaluation combined quantitative measures (Lighthouse Performance 96, Largest Contentful Paint 1.2 s, median warm query latency about 65 ms) with qualitative classroom testing, which drove six usability changes no automated metric detected. System testing in the final unit found and fixed one real regression, a silent end-of-round bell, growing the automated unit test suite from 24 to 39. The project demonstrates that a solo developer on managed infrastructure can deliver a secure, monetized, classroom-ready product in eight weeks, and that quantitative and qualitative evaluation catch different, equally necessary classes of defect.

ACKNOWLEDGEMENT

I would like to thank my course instructor, Dr. Sirisha Pavuluri, for the guidance and feedback throughout each unit of this capstone, as well as Dr. Sanjay Bhargava, Department Chair of the CS & MSIT department, and the University of the People for making this program accessible. I also want to acknowledge my ESL students and my co-workers who play-tested the game with their classes. Their very real struggles with speaking anxiety and their honest feedback are the whole reason this project exists and works.

LIST OF TABLES

LIST OF FIGURES

SYMBOLS AND ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
BaaS	Backend-as-a-Service
CI/CD	Continuous Integration / Continuous Deployment
CSV	Comma-Separated Values
ESL	English as a Second Language
FR / NFR	Functional Requirement / Non-Functional Requirement
HMAC	Hash-based Message Authentication Code
JWT	JSON Web Token
RLS	Row-Level Security
SSR	Server-Side Rendering
UI	User Interface

CONTENTS

INTRODUCTION

This report consolidates the first three units of my MSIT capstone into one document: the Unit 1 literature review, the Unit 2 problem definition, the Unit 3 architecture and design work, and a record of implementation progress so far. The project is 59 Seconds, a fast-paced ESL speaking game built to run on a single classroom computer and projector.

1.1 Background of the Study

I teach English as a Second Language, and the pattern I see most often is not students who cannot speak English, it is students who will not. Give a class a grammar worksheet and they will finish it quietly; ask them to describe something out loud in front of their peers and many of them freeze. Second-language research has a name for this: the affective filter, the idea that anxiety, low confidence, and boredom act like a wall that stops language input and output from flowing, no matter how much the student actually knows (Krashen, 1982).

Games are a well-known way around that wall, because when students focus on winning, they stop over-monitoring their own grammar. The inspiration here is the physical party game 30 Seconds, where one player describes words on a card while teammates shout guesses against a sand timer, loud, social, and nobody playing it is thinking about verb conjugation. 59 Seconds turns that experience into classroom software: a digital card of five words on the projector, one student describing, the whole class guessing, and a big 59-second timer counting down.

Two design convictions shape everything in this project. First, the game targets the classroom's existing hardware, one teacher PC and one projector, instead of assuming every student has a device. Second, the vocabulary is region-localized: a class in Seoul gets a Korea pack full of terms they actually recognize, instead of being stalled by obscure Western pop-culture references they have never heard of.

1.2 Problem Statement

The problem, as defined in Unit 2, sits at the intersection of web development, cloud databases, and educational technology. Traditional ESL speaking activities fail to motivate students because they are boring and anxiety-inducing, and the technology that schools bring in to fix this usually makes it worse in two specific ways. First, content is culturally generic: a Korean student who draws a flashcard about an American sitcom cannot practice describing it, because the barrier is not English, it is that they have no idea what the thing is. Second, delivery is device-isolated: nearly all modern educational apps assume a 1:1 smartphone or tablet setup, which many schools cannot afford, and which turns a speaking lesson into thirty students staring silently at their own screens, complete with notification distractions (Sung et al., 2016).

Formally: traditional ESL speaking solutions fail to motivate students because they rely on culturally generic content and isolating 1:1 smartphone setup that cause digital distractions. This project addresses the issue by deploying a projector-optimized, low-latency web application, 59 Seconds, that uses region-specific vocabulary decks and a collaborative single-screen layout to drive spontaneous, face-to-face English conversation.

Three stakeholder groups are affected. ESL instructors need low-prep speaking activities that engage a whole class at once. Language learners need a low-stakes, high-energy environment where they can speak without over-thinking mistakes. Educational institutions need affordable software that runs on hardware they already own, a laptop and a projector, rather than a device per child.

1.3 Objectives of the Study

The project goals were set in Unit 2 using the SMART framework, and they remain the yardstick for this report.

Specific: design and build a web-based speaking game platform on a serverless stack (React/TanStack Start, Supabase, Vercel, and Paddle sandbox) that displays five localized words per card with a 59-second countdown timer.

Measurable: the app must render eleven distinct regional packs, cleanly restrict access to seven gated decks per pack until a verified sandbox purchase unlocks them, and parse a correctly formatted CSV file on the admin side without errors.

Achievable: by leaning on Backend-as-a-Service infrastructure, the workload is compressed enough for a single developer to finish inside the timeframe.

Relevant: the application directly addresses the classroom need for high-energy oral fluency tools while satisfying the MSIT capstone criteria for full-stack architecture, security, and project tracking.

Time-bound: all milestones must be completed, tested, and ready for final submission by August 12, 2026.

1.4 Scope and Limitations

In scope: a high-contrast web UI optimized for large projector displays and mouse control; twelve culturally localized packs (Japan, Korea, USA, UK, South Africa, World, Ireland, Canada, China, India, Australia/New Zealand, and a Kids pack for ages six to seven added after classroom testing), each holding eight decks with the first deck free and anonymous; an authentication system (email and password, plus Google sign-in) used only when someone buys; a secure owner-only admin dashboard for uploading decks via CSV and managing deck names; a Paddle payment integration that ran in sandbox mode during development and switched to live production payments once the site deployed; a Supabase-backed database with Row-Level Security; and hosting with automatic deployments on Vercel.

Out of scope: native iOS/Android apps and real-time multiplayer features such as student phones acting as buzzers. The scope grew twice, deliberately. The catalog expanded from six launch packs to eleven because the CSV pipeline made adding packs a data task rather than a coding task, and a twelfth Kids pack was added after a co-worker's young learners found the existing packs too hard. Live production payments, originally out of scope, also shipped during the build. The key assumption is that classrooms have a working PC, a projector, and stable internet. The hard constraint is time: the whole project is built solo between June 18 and August 12, 2026.

1.5 Report Organization

Chapter 2 reviews the literature on speaking anxiety, game-based learning, classroom device models, and serverless architectures, then compares existing systems and identifies the research gap. Chapter 3 presents the system design: the architecture, the tools and methodologies, the data flows and module specifications, and the design diagrams. Chapter 4 documents the results: the delivered system, its outputs and screenshots, the evaluation metrics, and the system testing that closed the project. Chapter 5 discusses those results against the literature, covers limitations, and outlines future work, and Chapter 6 concludes. The appendices contain the database schema summary and the CSV import format.

LITERATURE REVIEW

2.1 Overview of Related Work

The theoretical starting point is Krashen's (1982) affective filter hypothesis: emotional factors, anxiety, low motivation, low self-confidence, act as a filter that blocks language acquisition even when the learner receives plenty of comprehensible input. In speaking, the filter shows up as students freezing because the social cost of a mistake feels too high. Any tool that wants students to speak more has to lower that filter first, which is why this project is a game rather than a drill.

The evidence that classroom games actually do this is strongest in the game-based student response literature. Wang and Tahir (2020) reviewed 93 studies of Kahoot! one of the most widely used classroom game platforms, and concluded it can have a positive effect on learning performance, classroom dynamics, student attitudes, and, importantly for this project, student anxiety. Just as useful for my purposes, their review catalogs the problems students report with projected classroom games: questions and answers that are hard to read on the projected screen, stressful time pressure, and not enough time to answer. Those complaints read like a design brief for 59 Seconds, and they directly informed three of my core decisions: the extremely large high-contrast text, the click-to-

edit timer that lets a teacher give a class more time, and the absence of any auto-advance, so the pace is always the teacher's call.

On the question of classroom device models, Sung et al. (2016) conducted a meta-analysis of 110 studies of mobile devices integrated into teaching and learning, finding a moderate overall positive effect on learning performance. I want to be fair to this finding: mobile learning clearly works in many contexts. But the same body of research also surfaces the practical costs of 1:1 device setups, procurement expense, classroom management overhead, and off-task distraction, and none of the reviewed models is designed for the thing an ESL speaking lesson needs most, which is thirty students looking at each other and talking. My project deliberately takes the opposite bet: one shared screen, zero student devices, and all the interaction happening face to face in the room.

Finally, on the technical side, Jonas et al. (2019) argue that serverless computing, including Backend-as-a-Service platforms, simplifies cloud programming the way high-level languages simplified assembly: the platform absorbs nearly all system administration and scales automatically. For a solo developer on an eight-week deadline, that argument is not abstract. Supabase (managed PostgreSQL, auth, and Row-Level Security) and Vercel (managed hosting and CI/CD) are what make the timeline feasible, and what make the app safe to trust in a live lesson, since there is no self-managed server for me to misconfigure.

2.2 Technologies and Platforms Reviewed

The second half of the review is about technology rather than pedagogy, because the choice of platform shaped what one person could realistically finish in eight weeks. I looked at four areas.

Frontend rendering

The game is a single-page React application, but I chose TanStack Start rather than plain client-side React because it supports server-side rendering. This matters in a classroom: a teacher often pastes a deck link straight into a school machine, and a page that arrives already populated instead of showing a loading spinner is the difference

between a lesson starting smoothly and a teacher waiting in front of thirty students. Server-side rendering also gives the pack pages real metadata for search engines, which is how most teachers will find the site.

Backend-as-a-Service and database-level security

Rather than writing my own backend I used Supabase, which provides PostgreSQL, authentication, and storage behind one API. The feature that decided it was Row-Level Security, a PostgreSQL mechanism that attaches access rules to the table itself rather than to application code (Supabase, n.d.). This matters because the usual way access control fails is that one forgotten check in one route leaks data, which is why broken access control sits at the top of the OWASP Top 10 (OWASP Foundation, 2021). Putting the rule in the database means a missing check in my code cannot expose paid content, because the database refuses the read regardless of what the application asks for.

Payments and the merchant-of-record model

Payments are handled by Paddle acting as merchant of record, which means Paddle is legally the seller and is responsible for collecting and remitting sales tax and VAT in each buyer's country. For a solo developer selling internationally that removes an enormous compliance burden, and it also means card details never touch my system. The security-critical piece is the webhook: Paddle signs each notification, and the documented practice is to verify that HMAC signature against the raw request body before trusting anything in the payload (Paddle, n.d.). That verification is what turns a purchase into a database fact in my system.

Testing and delivery practices

On testing, the literature draws a clear line between checking pieces and checking the product. Unit testing means validating individual components in isolation, using mocks or stubs to remove outside dependencies (Amazon Web Services, n.d.), while system testing evaluates the fully integrated system against its requirements and is the last verification before release (Hrupa, 2024; BrowserStack, 2025). Both matter, and Chapter 5 describes what happened when I had only the first one. Rana (2023) supplied the structure I used for writing individual test cases: preconditions, inputs, steps, expected

results. On delivery, the DevOps literature frames continuous integration and continuous deployment as the practices that let small changes reach users quickly and safely (Red Hat, 2025), and Sumo Logic (n.d.) describes the staged deployment patterns, previews, rollbacks and monitoring, that I ended up using in a simplified form.

2.3 Comparative Analysis of Existing Systems

Table 1 compares 59 Seconds against the three closest existing alternatives: Kahoot! and Quizlet Live as the dominant digital classroom game platforms, and the physical 30 Seconds board game as the direct inspiration.

System	Device model	Content localization	Speaking focus	Cost & setup friction
Kahoot!	One shared screen for questions, but every student answers on their own device	Teachers can author quizzes, but the format is multiple-choice recall, not description	Low - students tap answers; talking is optional	Freemium; requires a device per student or per team
Quizlet Live	Student devices grouped into teams	Teachers can build their own study sets	Low to medium - team chat happens, but the task is matching terms	Free tier; still requires student devices
30 Seconds (physical board game)	No devices; fully face-to-face	Fixed printed cards, region-locked to the edition you buy; heavily Western references in most editions	Very high - describing words aloud is the entire game	One-time purchase; cards wear out and cannot be edited or extended
59 Seconds (this project)	One teacher PC + projector; zero student devices	Region-specific packs (11 regions), extensible by CSV upload without code changes	Very high - the describe-and-guess loop is the whole game	First deck of every pack free with no account; $8.99 unlocks a full regional pack

Table 1. Comparative analysis of existing systems.

The positioning is clear from the table. The digital platforms have reach and editability but push interaction onto individual screens and quiz-style recall; the board game has exactly the right social mechanics but frozen, non-localized content. Nothing combines a single-shared-screen model with editable, region-localized, speaking-first content, the slot 59 Seconds fills.

2.4 Research Gap Identification

Two gaps emerge from the review. The first is the content gap: game platforms let teachers author content, but none ships with (or is structured around) region-localized vocabulary as a first-class concept, so a teacher in Seoul either builds every deck by hand or watches students stall on culturally foreign references. In 59 Seconds, the region pack is the core unit of content, and the admin CSV pipeline exists precisely so localized decks can be added as data.

The second is the screen gap: the education technology market overwhelmingly assumes students hold the screen. Wang and Tahir's (2020) review even documents the readability problems that arise when platforms designed for phones get projected onto a classroom wall as an afterthought. There is a shortage of software designed from the first pixel for a one-to-many projector layout, large type, huge timer, mouse-only control from the teacher's desk, where the students' job is to look up and talk. That is the gap this project's interface design targets directly.

SYSTEM DESIGN AND METHODOLOGY

3.1 System Architecture

Figure 1 shows the full system architecture. 59 Seconds is a serverless web application with three main layers plus two external services. The client layer is a React 19 application built on the TanStack Start framework with file-based routing; it runs in the browser on the teacher's PC and is displayed through the projector. The hosting layer is Vercel, which server-side renders the app (so deep links to individual decks work correctly) and runs the project's server functions: the admin upload and deck-editing logic, and the payment webhook endpoint at /api/public/paddle-webhook. The data layer is Supabase, which provides the PostgreSQL database, authentication, and Row-Level Security.

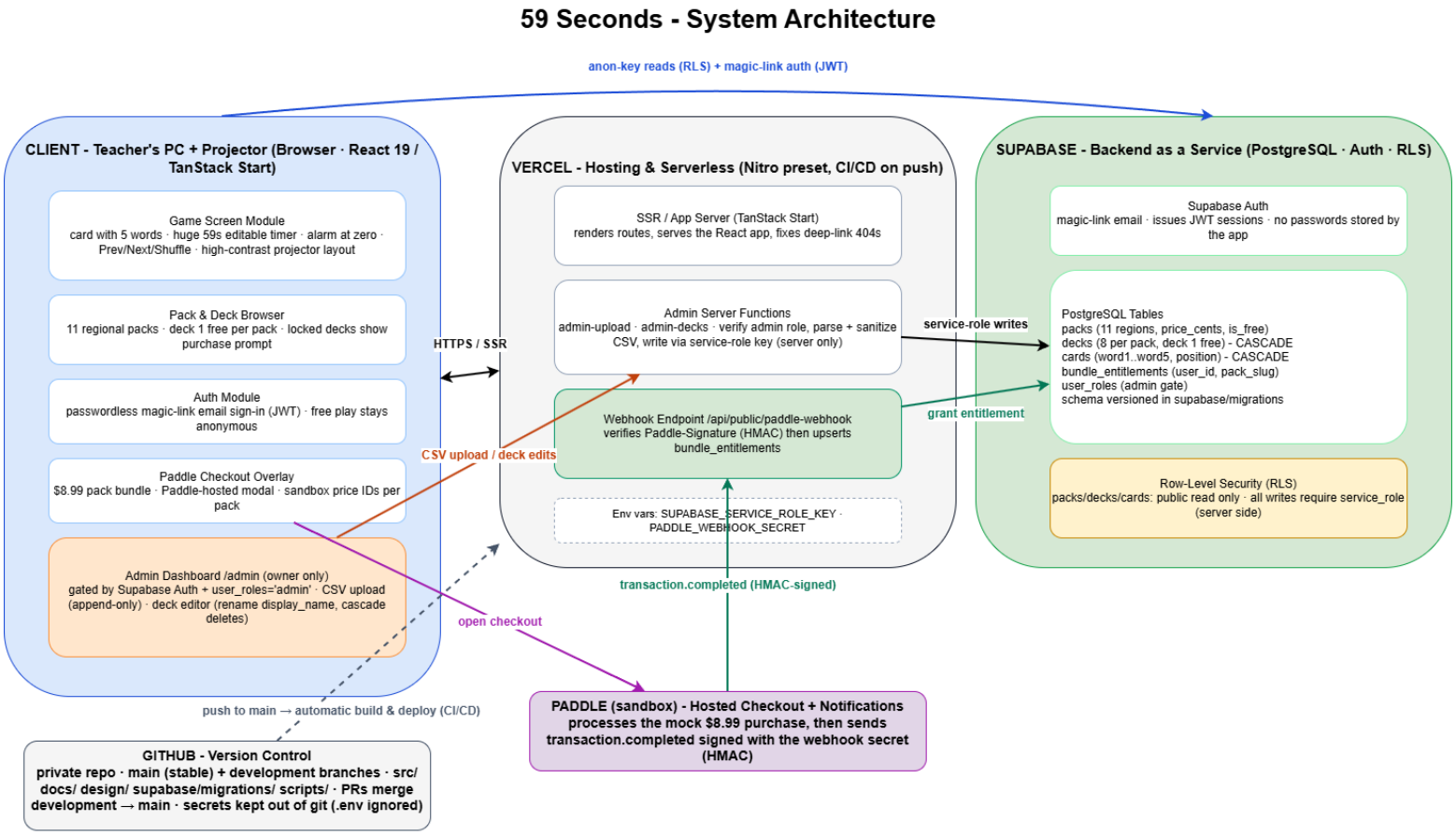

Figure 1. 59 Seconds system architecture. (Created in draw.io; the editable source is versioned in the repository.)

The two external services are Paddle and GitHub. Paddle hosts the checkout overlay (sandbox during development, live in production since mid-July) for the $8.99 pack bundles and, when a mock transaction completes, sends a transaction completed notification signed with an HMAC secret to the Vercel webhook endpoint (Paddle, n.d.). GitHub holds the code, documentation, and design files, and every push to the main branch triggers an automatic build and deployment on Vercel (Vercel, n.d.).

The most important architectural property is the trust boundary. The browser only ever holds Supabase's public anonymous key, and Row-Level Security policies restrict that key to read-only access on the content tables (Supabase, n.d.). Every write in the entire system, creating decks from a CSV, renaming a deck, granting a purchase entitlement, happens server-side using the service-role key, which exists only in server environment variables and never ships to the client. This means there is no client-side path to corrupt the content database or fake a purchase, which addresses the top categories of web application risk, broken access control chief among them (OWASP Foundation, 2021).

3.2 Methodologies and Tools Used

Development methodology

The build follows the four sequential phases planned in Unit 2, each ending in something testable: Phase 1 (architecture and setup) - repository, TanStack Start scaffold, Supabase schema; Phase 2 (UI and content pipeline) - projector-first interface and the admin CSV importer; Phase 3 (core game mechanics) - game loop, timer, and deck gating; Phase 4 (billing and deployment) - Paddle integration, end-to-end testing, Vercel deployment. I do not start a phase until the previous one runs, essentially incremental delivery scaled to one developer (Oguz, 2025). Tasks and risks are tracked on a GitHub Projects board.

Core game loop logic

The game logic is deliberately simple so that nothing can surprise a teacher mid-lesson. When a deck is opened, all of its cards are fetched in a single query and shuffled client-side, so every playthrough deals the cards in a new order without repeated network calls. Each card renders its five entries at once, with parenthetical hints displayed smaller and lighter than the main words. The 59-second timer is click-to-edit, a teacher can set 30, 90, or any value that suits the class, and when it reaches zero it plays an alarm but does not advance the card. Navigation is strictly manual: Previous, Next, and Shuffle buttons, all mouse-sized for use from the teacher's desk. Nothing moves unless the teacher moves it.

Security methodology

Three mechanisms carry the security design, all specified in Unit 2 before implementation. First, Row-Level Security on every table: packs, decks, and cards are publicly readable and writable only by the service role, while entitlements and roles are locked down entirely (Supabase, n.d.). Second, authentication is outsourced to Supabase Auth, with email-and-password and Google OAuth sign-in, so the app never implements its own credential handling, and sessions are standard JWTs. Third, purchase state is never trusted from the frontend: the only code that can grant an entitlement is the

webhook handler, which verifies the HMAC Paddle-Signature header before touching the database (Paddle, n.d.). Editing browser state cannot unlock anything, because unlocking is a database fact, not a client flag. Finally, the CSV importer sanitizes incoming text against script injection, since the admin upload is the one place user-supplied files enter the database.

Content pipeline

Content management had to work without code changes, and it does. The admin uploads a UTF-8 CSV in which each row carries a deck slug (for example KoreaS01Deck3) and five words. The importer validates the slug pattern, sanitizes the text, and appends new decks and cards, it is append-only by design, so an accidental re-upload can never silently overwrite live content; replacing a deck is an explicit delete-then-import. Deck accent colors are assigned automatically from position, and curated display names can be edited later in the deck editor. A small Node script handles the deck artwork separately, center-cropping source images to a consistent 1024×768 format.

3.3 Algorithmic Design

Four pieces of logic do the real work in 59 Seconds. None of them is complicated on its own, but each one had a specific requirement behind it, and three of the four ended up being rewritten during the project because testing or a classroom found a problem with my first version.

Card shuffling

Cards are randomized with the Fisher-Yates shuffle, which walks the array backwards and swaps each element with a randomly chosen earlier one. I picked it because it is provably unbiased: every possible ordering is equally likely, which naive shuffling approaches do not guarantee. I implemented it as a pure function that copies its input rather than reordering it in place, because mutating state directly causes bugs in React.

```
function shuffle(arr):
    a = copy of arr
    for i from length(a) - 1 down to 1:
        j = random integer in 0..i
        swap a[i] and a[j]
    return a
```

A second shuffling rule was added later. A co-worker noticed during a lesson that the same cards were reappearing before the deck had been used up, which is annoying in a classroom because students recognise a repeat immediately. The fix was to treat the deck as a lap: the order is exhausted completely before any card can come round again, and the reshuffle for the next lap keeps the current card in place so the screen does not change underneath the teacher.

The countdown timer

The timer is the most visible thing on screen, so it has more rules than it looks like it needs. It ticks every 100 milliseconds rather than every second so the drain bar animates smoothly, subtracts a fixed step, rounds to two decimal places so repeated floating-point subtraction cannot drift, and clamps at zero so it can never display a negative number. A separate rule decides whether the end-of-round bell should ring, and Chapter 4 explains why that decision had to be pulled out into its own function.

```
function tickRemaining(remaining, step = 0.1):
    next = round(remaining - step, 2)
    if next < 0: return 0
    return next

function shouldRingBell(remaining, alreadyRang):
    return remaining <= 0 and not alreadyRang
```

CSV parsing and row validation

Content enters the system as a CSV file, so the import logic has to be forgiving about how the file was written but strict about what reaches the database. Parsing tolerates three different spellings of the deck-name column, trims whitespace everywhere, and silently drops completely blank rows. Validation then runs twice, once in the browser to give the admin a preview and once again on the server before anything is written, and

returns either a cleaned row or a refusal with a stated reason so the importer can report exactly which rows it skipped and why.

```
function validateUploadRow(row):
    slug = trim(row.deckSlug)
    if slug is empty:
        return { valid: false, reason: 'Missing deck name' }
    words = [trim(w) for w in row.word1..word5]
    if any word is empty:
        return { valid: false, reason: 'Missing one or more words' }
    return { valid: true, normalized: { slug, words } }
```

Word parsing on the card

Each word entry can carry a category hint and a translation, and content authors do not write them in a consistent order. Rather than forcing one format, the parser decides by content: a group in brackets containing any non-Latin character is treated as a translation, a group containing only Latin characters is treated as a category badge. That single rule means "Word (Hint) (Translation)" and "Word (Translation) (Hint)" both render identically, which saved a great deal of tedious cleanup across 14,400 words.

Entitlement resolution

Deciding whether someone may open a deck is the one algorithm with money attached, so it deliberately does the least clever thing possible. The browser never decides. Every deck read goes to the database, Row-Level Security allows the row through only if the deck is free or the signed-in user has a matching entitlement row, and entitlement rows are written exclusively by the verified payment webhook. There is no client-side flag to flip.

3.4 Data Flow and System Components

Three data flows cover everything the system does. The play flow is anonymous and read-only: the browser queries packs and decks with the anon key, the teacher picks a free (or owned) deck, the cards arrive in one query, and from that point the entire game, timer, alarm, navigation, runs locally in the browser, which is why a mid-game network hiccup cannot interrupt a round. The purchase flow starts when a user hits a locked deck: they sign in, the Paddle overlay opens with the pack's sandbox price, Paddle processes

the mock payment and fires the signed webhook, the Vercel endpoint verifies the signature and upserts a row into bundle entitlements, and the pack's remaining seven decks unlock. The admin flow is the owner-only path: an authenticated admin (checked against the user roles table) uploads a CSV or edits deck names through server functions that hold the service-role key.

Table 2 lists the functional requirements the system was built against, Table 3 lists the non-functional requirements with the design decisions they drove, and Table 4 specifies each module's inputs, outputs, and methodology.

ID	The system shall…
FR1	Display a game card of 5 words in large, high-contrast text readable from the back of a classroom.
FR2	Run a 59-second countdown (click-to-edit) that sounds an alarm at zero without auto-advancing the card.
FR3	Provide manual Previous, Next, and Shuffle controls operated by mouse only.
FR4	Let anyone browse 11 regional packs and play the first deck of every pack free, with no account.
FR5	Sign users in with a passwordless email magic link when they choose to buy.
FR6	Unlock all 8 decks of a pack after a verified sandbox purchase, via the signed Paddle webhook.
FR7	Let the admin add new decks and cards by uploading a CSV file, with no code changes.
FR8	Let the admin rename deck display names and delete decks or whole packs (with confirmation).

Table 2. Functional requirements.

Category	Requirement	Design decision it influenced
Performance	A free game must start in under 5 seconds from landing on the site.	SSR on Vercel; no sign-up wall on free content; one query per deck.
Usability	Fully usable on a projector from the back of a room, mouse-only.	Dark high-contrast theme; very large text, buttons, and timer; hints rendered smaller than the main words.
Reliability	The game must not break mid-lesson.	No auto-advance; timer and alarm run locally in the browser; the payment loop was tested end-to-end before relying on it.
Scalability	Adding content or users must not require re-engineering.	New packs are data (a CSV import), not code; Supabase and Vercel scale on managed tiers.
Security	No client-side path to modify data or fake a purchase.	RLS blocks client writes; service-role key stays in server env vars; entitlements written only by the verified webhook; CSV input sanitized.

Table 3. Non-functional requirements and the design decisions they influenced.

Module	Inputs	Outputs	Methodology
1. Game Engine	Deck slug; mouse clicks; timer edits	Card of 5 words; countdown; alarm at zero	React state machine; cards fetched once, shuffled client-side; click-to-edit timer; no auto-advance.
2. Pack & Deck Browser	packs/decks queries; user's entitlements	Pack grid (11 regions); deck grid with lock states	Anon-key reads under RLS; deck 1 flagged free; entitlement row unlocks the rest.
3. Authentication	User's email address	JWT session (or nothing, for anonymous play)	Supabase Auth magic link; passwordless; only needed to buy.
4. Payments & Entitlements	Pack slug + price ID; Paddle transaction.completed	bundle_entitlements row; unlocked pack	Overlay checkout; HMAC-verified webhook; server-side upsert.
5. Admin Content Manager	UTF-8 CSV; rename/delete actions	New decks/cards; display_name updates; cascade deletes	Auth + user_roles gate; append-only sanitized import; inline delete confirmation.
6. Data Layer	SQL queries from the modules above	packs, decks, cards, entitlements, roles rows	PostgreSQL with RLS; schema versioned in supabase/migrations.

Table 4. Module specifications (inputs, outputs, methodology).

3.5 Design Diagrams (ER Diagram)

Figure 2 shows the entity-relationship diagram of the database. The content side is a straightforward hierarchy: one pack has many decks (eight per pack in practice), and one deck has many cards, with ON DELETE CASCADE on both foreign keys so removing a pack cleans up everything beneath it in a single operation - which is exactly what the admin deck editor relies on. The account side hangs off Supabase's managed auth.users table: bundle_entitlements records which user owns which pack (one row per unlocked pack, written only by the webhook, and carrying the Paddle transaction identifiers for auditability), and user_roles marks which user is the admin.

The two sides connect logically through the pack slug, which keeps entitlements readable and debuggable - you can see at a glance that a user owns "korea" - while the content hierarchy stays keyed on integer IDs.

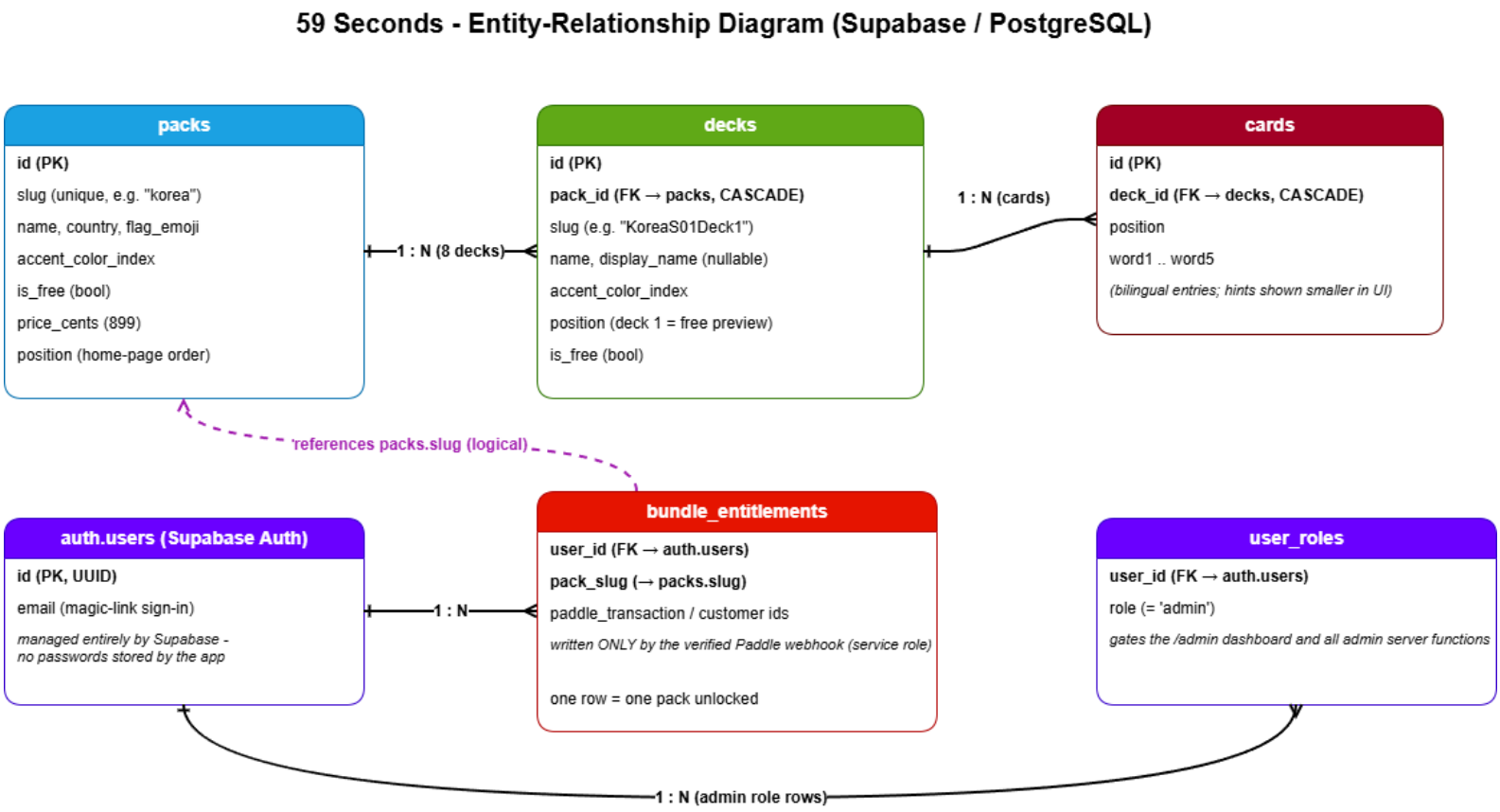

Figure 2. Entity-relationship diagram of the Supabase (PostgreSQL) schema.

Figures 3 to 6 record the four main screens as they stood at the end of the design phase; the final production versions appear in Chapter 4.

Figure 3. Home Screen (design-phase build).

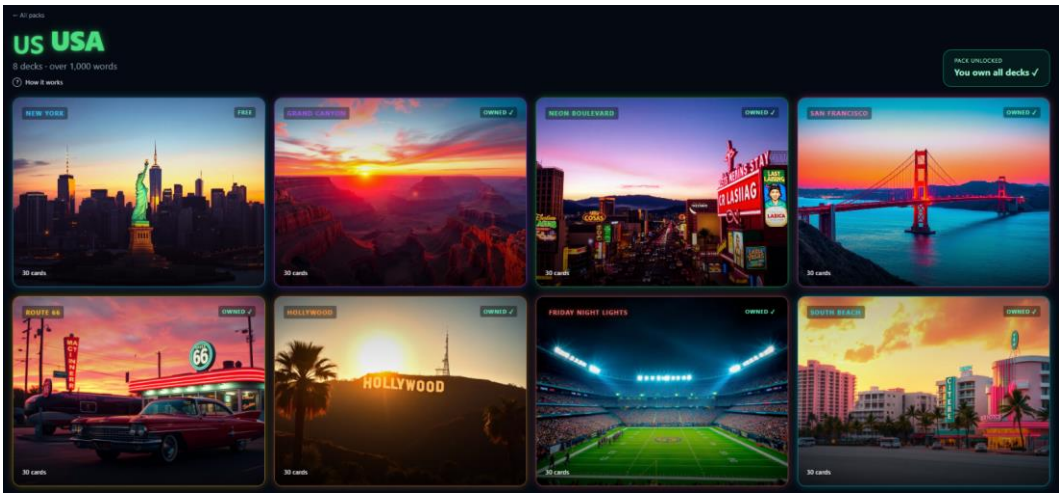

Figure 4. Pack Screen (design-phase build).

Figure 5. Game Screen (design-phase build).

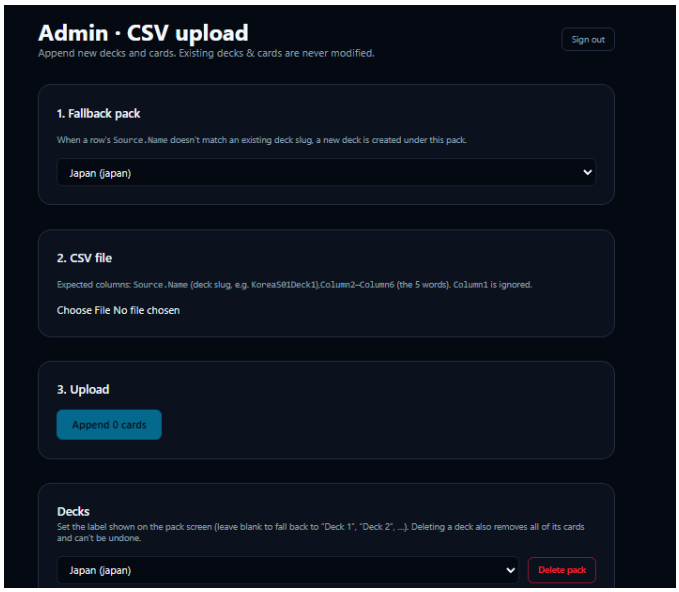

Figure 6. Admin Dashboard (design-phase build).

3.6 Design-Phase Progress

By the end of the design phase the build was already ahead of its Gantt baseline: the core game, the content pipeline, and the complete sandbox payment loop were working, which were originally later-phase items. Table 5 records the milestones as they stood at that point, and Table 6 records the four challenges that cost real time during the

first half of the project, because each one changed how I work. Both tables are kept here as the honest historical record; Chapter 4 reports where the finished system ended up.

Milestone	Status / evidence
Repository, scaffold, and database schema	Done. Private GitHub repo with main + development branches; Supabase schema (packs, decks, cards, bundle_entitlements, user_roles) versioned as migrations.
Projector-first UI	Done. Dark high-contrast theme, oversized text and controls, huge editable timer; pack flags and per-pack accent colors on the home screen.
Core game loop	Done. Shuffled cards, five entries per card with smaller parenthetical hints, 59 s click-to-edit timer with alarm at zero, manual Prev/Next/Shuffle.
Admin content pipeline	Done. Role-gated /admin with append-only CSV import and a deck editor (rename display names, cascade deletes with inline confirmation).
Eleven-pack catalog	Done structurally. All 11 packs exist with 8 deck slots each, deck 1 free, sandbox price IDs wired; five newer packs still await card CSVs and some artwork.
Deck artwork pipeline	Done. Script center-crops source images to 1024×768; images wired for 8 of 11 packs so far.
Authentication and deck gating	Done. Magic-link sign-in; anonymous free play; locked decks prompt purchase.
Sandbox payment loop	Done and verified end-to-end on June 30, 2026: checkout → signed webhook → signature verified (HTTP 200) → entitlement written → pack unlocked in the UI.

Table 5. Milestones achieved (as of end of Unit 3).

Challenge	What happened and how it was resolved
Character-encoding corruption	A Korea deck exported as plain ANSI CSV turned every Korean character into literal question marks - unrecoverable once imported. The corrupted cards were deleted and the deck is being re-imported from a proper UTF-8 CSV. Lesson adopted: every CSV must be saved as "CSV UTF-8" before upload, and this is now a documented rule of the pipeline.
Webhook secret format confusion	Paddle's dashboard shows a notification-destination ID that looks like a secret but is not the signing secret; using it caused every webhook to fail signature verification with HTTP 401. Resolved by copying the full secret key value. The debugging session is why signature verification is now the best-understood code path in the project.
Testing webhooks against localhost	Paddle cannot reach a local dev server, so the purchase loop could not be tested naively. Resolved with a temporary Cloudflare tunnel exposing the local server over HTTPS, plus replaying events from Paddle's delivery log instead of re-purchasing each time.
Secrets committed to git history	Early commits included the .env file, meaning the Supabase service-role key exists in the repository's history even after .env was gitignored. Mitigation: the key is being rotated in the Supabase dashboard, which makes the historical value useless, before the repository is shared for grading.

Table 6. Key challenges faced and how they were resolved.

RESULTS AND EVALUATION

This chapter covers what the project actually produced across Units 4, 5 and 6: where the implementation ended up, what the system now puts on screen, and what happened when I measured it. Where the finished system differs from what I planned back in Units 1 to 3, I have said so plainly instead of quietly moving the goalposts, because a couple of those differences turned out to be among the more interesting results.

4.1 Implementation Overview

59 Seconds is finished, deployed, and being used in real lessons. The app is live at https://59seconds.live on its own domain, serving twelve region-specific packs to real users, taking real payments, and getting played by university students and by primary-aged children at the British Council. That is a big shift from where things stood at the end of Unit 3, when it only ran on my own machine with pretend payments.

The build followed the four phases I set out in Unit 2, and each of the last three units added a different layer. Unit 4 produced the first working version, along with the understanding of continuous integration and deployment that shaped how the project ships. Unit 5 tightened up the core logic: I pulled the algorithms that run the game out into modules I could actually test, set up a unit test suite with Vitest, and started tagging milestones as proper releases. Unit 6 brought every module together into one system, measured it against both quantitative and qualitative metrics, and pinned down how it gets deployed and configured.

Scope evolution since Unit 3

The finished system goes beyond what I promised in Unit 2 in four places, and moves away from it in one. Table 7 sets these out honestly, because a capstone that quietly changes its own targets is harder to mark than one that says what moved and why.

Area	Planned (Units 1-3)	Delivered (end of Unit 6)
Payments	Paddle sandbox only; production billing explicitly out of scope	Live production payments since 17 July 2026. A real purchase and a real refund were both processed end to end, with entitlement granted and then revoked by the webhook.
Content catalogue	Eleven region packs	Twelve packs. A Kids pack for ages six to seven was added after classroom testing showed the existing packs were too difficult for young learners.
Hosting	Vercel deployment planned	Deployed on Vercel behind the custom domain 59seconds.live, purchased specifically because Paddle will not approve shared subdomains for live accounts.
Testing	Manual verification only	A Vitest suite of 24 automated unit tests across three files, all passing.
Authentication	Passwordless email magic link	Email and password sign-in plus Google OAuth through Supabase. Magic-link-only sign-in proved awkward for teachers buying on a school machine, and Google sign-in removed a step at the point of purchase.

Table 7. Planned scope versus delivered system.

The authentication change is the only real departure rather than an expansion, and I want to be clear that it was a decision I made while building, not something I forgot. The security properties I claimed in Unit 3 all still hold: sessions are still standard JSON Web Tokens issued and checked by Supabase, the app still never handles a password itself, and whether someone has paid is still never taken on trust from the browser.

Module integration

Five modules now work as one system. The game engine draws cards, runs the timer and keeps team scores. The pack and deck browser lists the catalogue and works out what a given visitor is allowed to open. Authentication issues and checks sessions. The payments and entitlements module runs checkout and handles webhooks. The admin content manager imports and edits content. What holds them together is a single Supabase PostgreSQL database and one strict rule about who is allowed to write to it.

That rule is the spine of the whole thing. Row-Level Security makes the content tables readable by anyone using the browser's publishable key, but every single write in

the application, whether that is creating decks from a CSV, renaming a deck, or granting someone access after a purchase, runs server-side with the service-role key. Two separate Supabase clients enforce this in the code itself. The browser client keeps its session and refreshes tokens, because a signed-in teacher should stay signed in. The server client deliberately turns off session storage, persistence and token refresh, because it handles exactly one request and is then thrown away. Neither key can do the other one's job.

Inside the game engine, three pieces of logic do most of the work, and all three were tightened up during Unit 5. Card order comes from a Fisher-Yates shuffle written as a pure function that copies its input instead of changing it, which gives a fair ordering and avoids the state-mutation bugs that shuffling in place causes in React. The timer ticks every hundred milliseconds, stops at zero rather than going negative, fires its alarm exactly once using a reference guard so a re-render cannot set it off twice, and ends the round automatically when it runs out. Team scores live entirely in browser state and can be reset whenever the teacher wants.

The important consequence of building it this way is that once a deck's cards have been fetched in one query, the whole round runs locally in the browser. A network hiccup partway through a lesson cannot stall the game, which was one of the reliability requirements I wrote in Unit 2.

The purchase flow is the most useful example of the modules working together, because it crosses every boundary the system has. A teacher opens a locked deck, signs in, and gets a Paddle checkout overlay. Paddle handles the money and, once the payment goes through, sends a signed notification to an endpoint on my server.

That endpoint refuses the request at four separate points before it will write anything. It returns 500 if the signing secret is missing, 403 if the request did not come from one of Paddle's addresses, 401 if the HMAC signature does not check out against the raw request body, and 400 if the payload will not parse. Only a request that gets past all four causes an entitlement row to be written, and that write is an upsert keyed on the Paddle transaction ID, so if Paddle sends the same notification twice the user does not end up with two entitlements. Refunds and chargebacks come through the same endpoint as adjustment events and delete the row, which takes the access away again automatically.

4.2 Output and Screenshots

The clearest way to show the modules really are joined up is to follow one piece of content the whole way from a spreadsheet on my desk to a card on a classroom wall, because that path goes through the admin module, the database and the game engine in turn.

	A	B	C	D	E	F	G
1	Source.Name	Column1	Column2	Column3	Column4	Column5	Column6
2	JapanS01Deck1	Card 1	Karaoke (Activity) (カラオケ)	Futuristic (Adjective) (未来的)	Vinland Saga (Anime) (ヴィン	Ikigai (Life Purpose) (生きがい)	Silver (Color) (銀色)
3	JapanS01Deck1	Card 2	Yakimeshi (Fried Rice) (焼き飯)	Mushroom (Vegetable) (きのこ)	One Punch Man (Anime) (ワン	Wabi-sabi (Imperfect Beauty) (侘(	Omotesando (Neighborhood
4	JapanS01Deck1	Card 3	River (Nature) (川)	Konbini (Convenience Store) (コン	Melon Pan (Sweet Bread) (メ	Roblox (Video Game) (ロブロック	Calling (Action) (電話してい
5	JapanS01Deck1	Card 4	Soft (Adjective) (柔らかい)	Buddhism (Religion) (仏教)	Garigari-kun (Ice Pop) (ガリガ	Thinking (Action) (考えている)	Kashiwa Mochi (Rice Cake) (柏
6	JapanS01Deck1	Card 5	Volleyball (Sport) (バレーボール	Oyakodon (Rice Bowl) (親子丼)	Flying (Action) (飛んでいる)	Naoshima (Art Island) (直島)	Maneki Neko (Lucky Cat) (招
7	JapanS01Deck1	Card 6	Textbook (Object) (教科書)	Smiling (Action) (微笑んでいる)	Cake (Dessert) (ケーキ)	The Witcher (Video Game) (ウィッ	Korokke (Croquette) (コロッ
8	JapanS01Deck1	Card 7	Pizza (Food) (ピザ)	Eel (Fish) (うなぎ)	Capsule Hotel (Hotel) (カプセ	Climbing (Action) (登っている)	Hi-Chew (Candy) (ハイチュウ
9	JapanS01Deck1	Card 8	Tap (Action) (タップ)	Tetris (Video Game) (テトリス)	Sword Art Online (Anime) (ソ	Miyamoto Musashi (Swordsman)	Long (Adjective) (長い)
10	JapanS01Deck1	Card 9	Doctor (Job) (医者)	Hokkaido Dog (Dog Breed) (北海道	Helicopter (Vehicle) (ヘリコプ	Pigeon (Bird) (鳩)	Chopsticks (Utensil) (箸)

Figure 7. Input: a UTF-8 CSV of Japan deck rows before upload.

Figure 7 shows the input format. Each row carries a deck slug in the Source.Name column and five word entries in Column2 through Column6. Figure 8 shows what comes out the other end. The entry "Karaoke (Activity) (カラオケ)" has been split into three separate parts on screen, the headword itself, an uppercase category badge, and the Japanese translation set smaller and without the glow on a second line, and it now shows up as card 23 of 30 in the Akihabara deck alongside six team scoreboards and the timer. I did not touch any code at any point in that journey.

Figure 8. Output: the same rows rendered as a playable card with team scoring.

The parsing rules behind that are deliberately relaxed about order, because whoever writes the content does not reliably put hints and translations in the same sequence every time. Anything in square brackets at the end is always treated as a category badge. Anything in brackets containing a non-Latin character is treated as a translation. Anything in brackets with only Latin characters is treated as a badge. So "Word (Hint) (Translation)" and "Word (Translation) (Hint)" both come out looking the same.

A few deliberate interface decisions are visible in Figure 8, and each one comes from a classroom constraint rather than from how I wanted it to look. The timer sits top right with a draining bar, put there so it stays visible from the back rows even when students crowd the front. It always shows two digits, so counting down from ten to nine does not shift the layout, and it turns white-hot as time runs low because that reads clearly against all ten of the accent colors the packs cycle through.

Navigation is arrows only, set as tall columns either side of the card so a teacher gets one big click target. Team scoreboards run down both sides rather than along the bottom, which leaves the word list the vertical space it needs. A row of keyboard shortcuts sits under the main control for teachers who would rather not use the mouse at all. For decks with translations, a resize observer measures the word list against the height of the card and scales it down if it would otherwise overflow, which stops the first and last entries of a Japanese or Korean deck getting cut off.

Figures 9 and 10 show the code behind the two most important cross-module interactions I described above: the webhook handler that turns a payment into a fact in the database, and the client split that keeps the browser away from the privileged key.

```
export const Route = createFileRoute("/api/public/paddle-webhook")({
  server: {
    handlers: {
      POST: async ({ request }) => {
        const secret = process.env.PADDLE_WEBHOOK_SECRET;
        if (!secret) {
          return new Response("Webhook not configured", { status: 500 });
        }
        if (!(await sourceIpAllowed(request))) {
          return new Response("Forbidden", { status: 403 });
        }
        const rawBody = await request.text();
        const sig = parsePaddleSignature(request.headers.get("paddle-signature"));
        if (!verify(rawBody, sig, secret)) {
          return new Response("Invalid signature", { status: 401 });
        }

        let event: any;
        try {
          event = JSON.parse(rawBody);
        } catch {
          return new Response("Invalid JSON", { status: 400 });
        }
        const { supabaseAdmin } = await import("@/integrations/supabase/client.server");
        const { error } = await supabaseAdmin
          .from("bundle_entitlements")
          .upsert(
            {
              user_id: userId,
              pack_slug: packSlug,
              paddle_transaction_id: transactionId,
              paddle_customer_id: customerId ?? null,
              paddle_price_id: priceId ?? null,
            },
            { onConflict: "paddle_transaction_id" },
          );
        if (error) {
          console.error("paddle-webhook insert failed", error);
          return new Response("DB write failed", { status: 500 });
        }

        return new Response("ok", { status: 200 });
      },
    },
  },
});
```

Figure 9. The Paddle webhook handler: layered rejection, then an idempotent entitlement upsert.

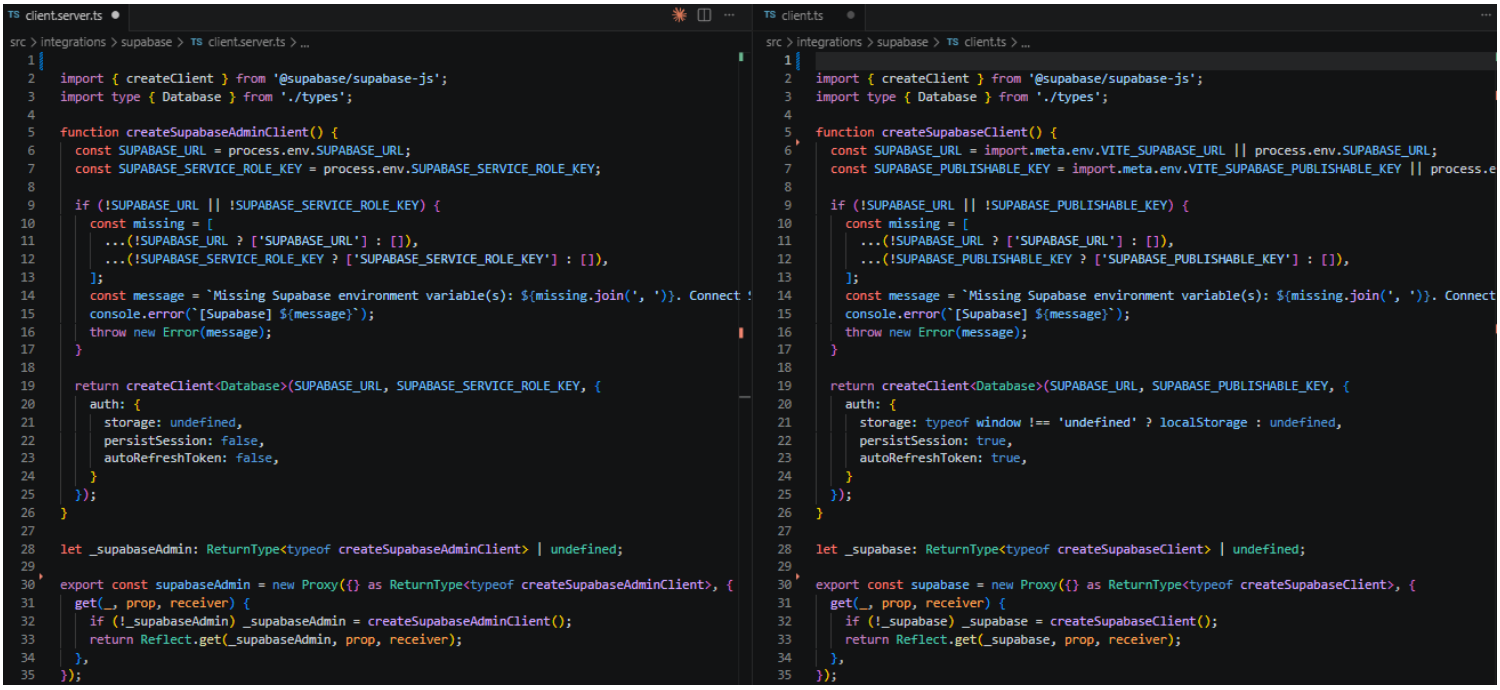

Figure 11 shows how the interface gets its data. The route loader fetches a query during server-side rendering, and the React component then asks for that same query key, reading it out of the cache instead of making a second request. This is why a link straight to a particular pack arrives with everything already on it rather than showing a loading state, which matters when a teacher pastes a URL into a classroom machine.

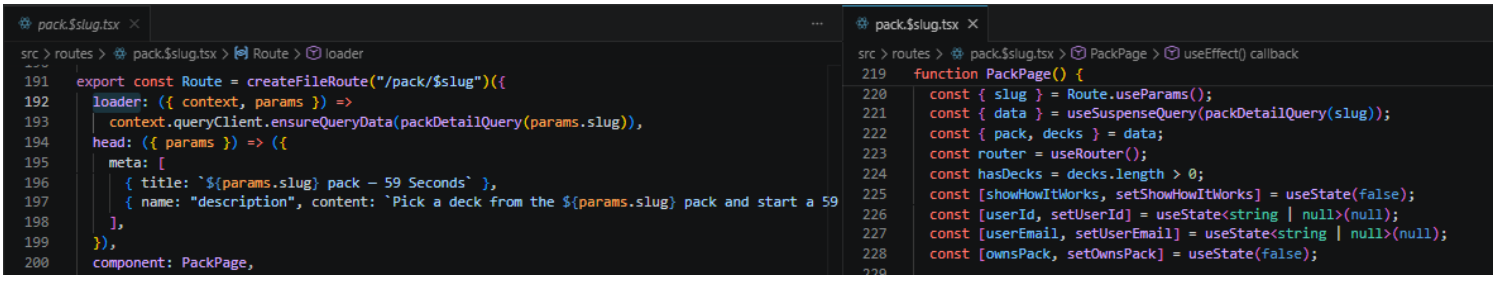

Figure 11. UI-to-backend linkage: the route loader prefetches the query the component consumes.

Figures 12 and 13 show two of the core algorithms I tightened up in Unit 5. The card shuffle uses Fisher-Yates, written as a pure function that copies its input and hands back a new array instead of changing the original, which both guarantees a fair ordering and keeps React out of trouble. The row validator returns one of two things, either a

cleaned-up row or a refusal with a stated reason, which is what lets the importer tell me exactly which rows it skipped and why.

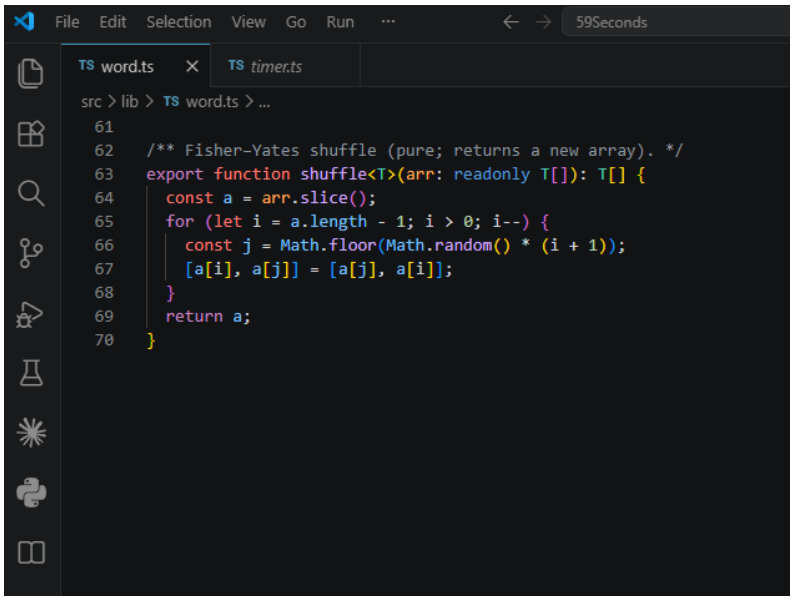

Figure 12. The Fisher-Yates shuffle in src/lib/word.ts.

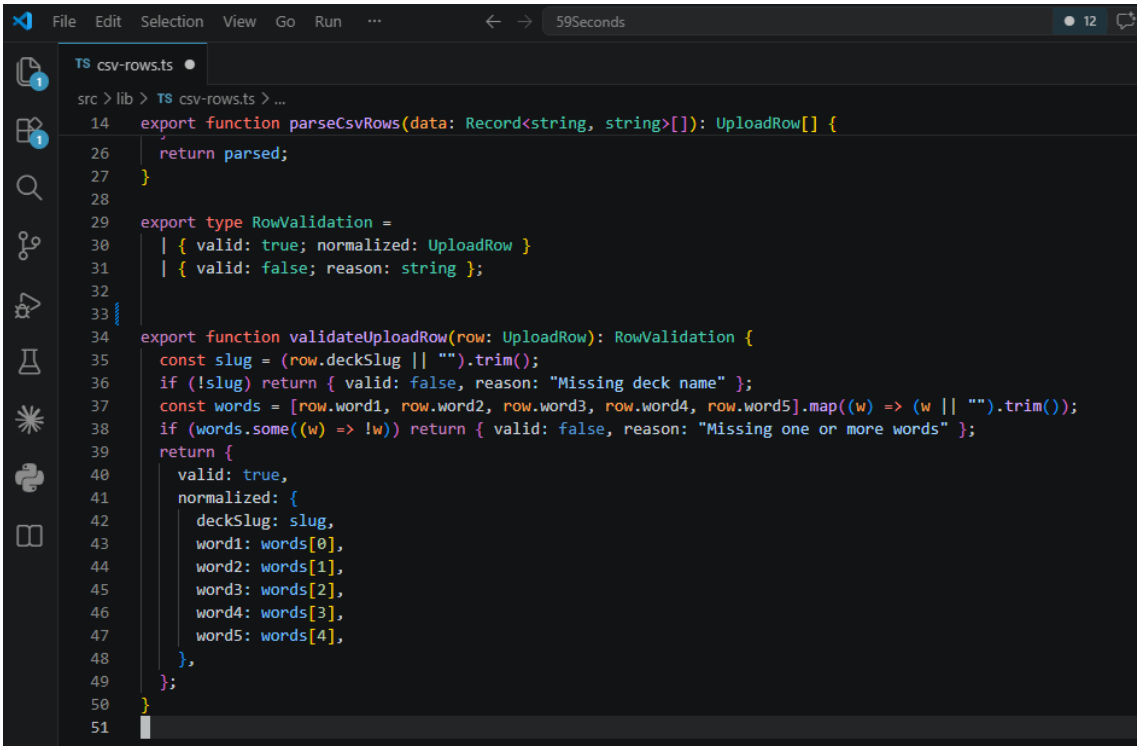

Figure 13. Row validation in src/lib/csv-rows.ts.

Figures 14 to 17 show the four screens a user actually meets. Figures 14 and 15 together make up the home screen, split across two images because twelve packs will not fit legibly into one, with each pack shown as a flag tile in its own accent color. Figure 16 shows a pack opened up to its eight decks, with the free one clearly marked off from the seven a purchase unlocks. Figure 17 shows the admin importer previewing a file before anything is committed, which is the screen behind the content pipeline described above.

Figure 14. The home screen with the first four region packs.

Figure 15. The home screen with the other eight region packs.

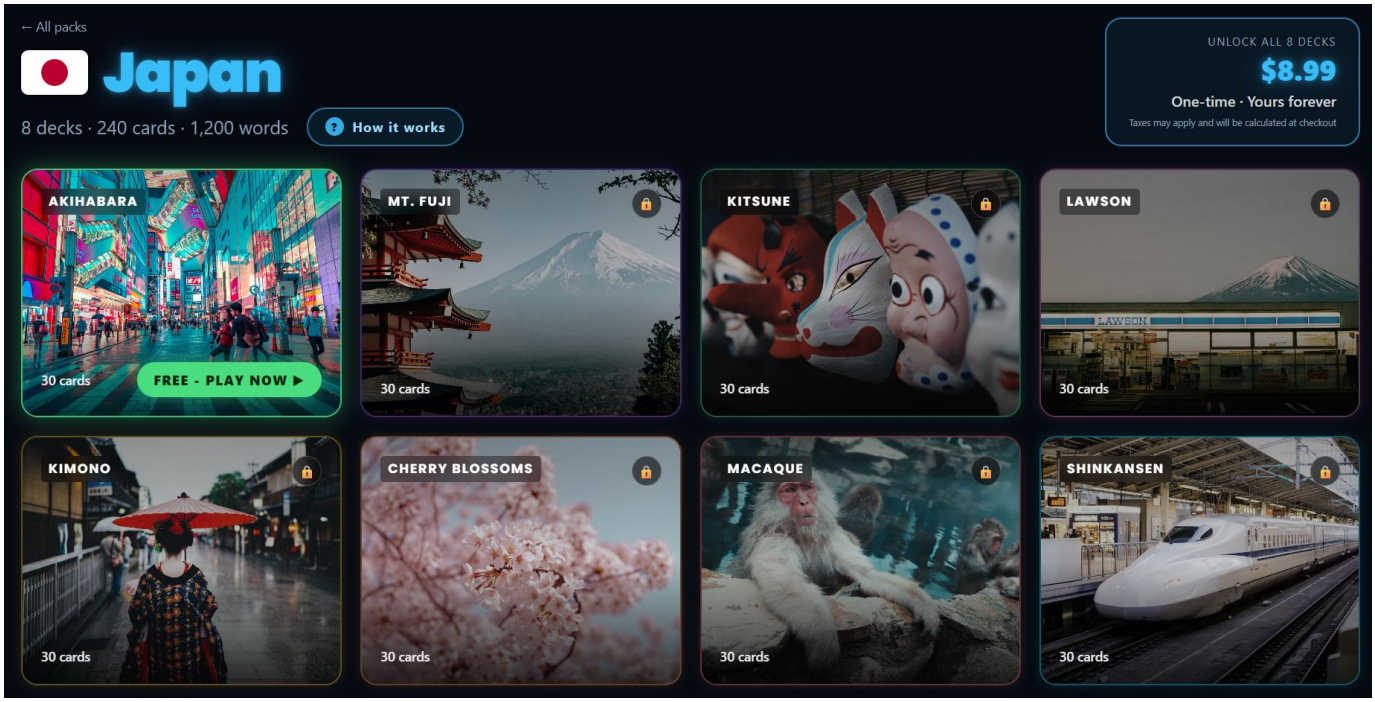

Figure 16. A pack screen: eight decks, with the free deck highlighted and locked decks dimmed.

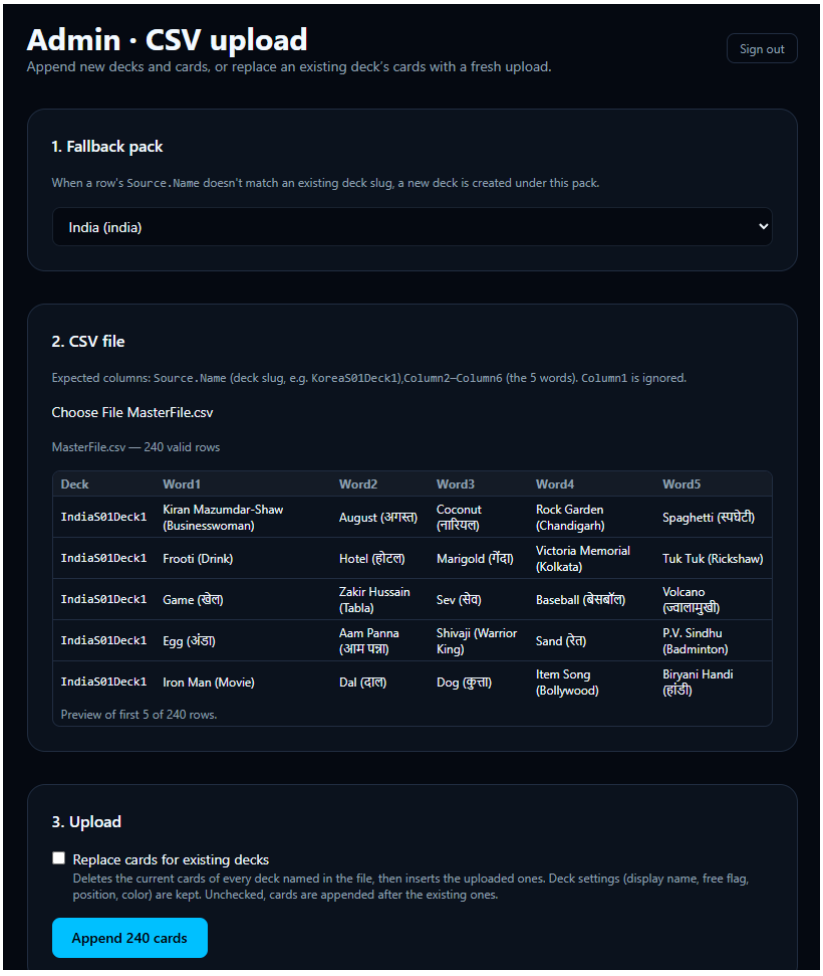

Figure 17. The admin CSV upload screen previewing rows before import.

4.3 Evaluation Metrics

I evaluated the system with both quantitative and qualitative measures, on the basis that the two answer different questions and work best together (LaunchNotes, 2023). Everything below was measured against the live site rather than a development build.

Quantitative results

A Lighthouse 12.8.2 audit in desktop mode gave scores of 96 for Performance, 100 for Accessibility, 93 for Best Practices and 100 for Search Engine Optimization. The same run reported a First Contentful Paint of 0.8 seconds, a Largest Contentful Paint of 1.2 seconds, Time to Interactive of 1.2 seconds, a Speed Index of 1.3 seconds, Total Blocking Time of 0 milliseconds and Cumulative Layout Shift of 0.001. Time to first byte, measured separately with repeated curl requests, came in between 0.46 seconds on the home and pricing pages and 0.74 seconds on the pack page.

I measured the database directly against the Supabase REST API. Listing all the packs took between 53 and 82 milliseconds once warm, deck metadata joined to its parent pack took 55 to 75 milliseconds, and pulling all thirty cards of a deck took 62 to 82 milliseconds, so the median warm response was somewhere around 65 milliseconds. The very first request of a session took 1.10 seconds because of cold start and the TLS handshake, but everything after that stayed under 100 milliseconds.

Build and code metrics give a third angle. A full production build finishes in 55.3 seconds and transforms 394 modules. The main client bundle is 566 kilobytes raw but 167.61 kilobytes gzipped, with stylesheets adding another 17.72 kilobytes gzipped. Route-level code splitting does what it is supposed to: the play route adds 6.60 kilobytes gzipped, the pack route 4.96 kilobytes, and the 13.12-kilobyte admin chunk never gets sent to ordinary players at all. At the end of Unit 6 the test suite ran 24 tests across three files in 1.31 seconds with nothing failing; it has since grown to 39, for reasons covered in Section 4.4. I checked the content scale by counting rows in the production database rather than estimating: twelve packs, ninety-six decks, 2,880 cards and 14,400 individual words.

These build numbers matter more here than they would for a normal web app, because the target is a school. Plenty of classrooms run on shared connections of unpredictable quality, and a teacher who has set aside thirty minutes for an activity will give up on a tool that visibly takes time to appear. A gzipped main bundle of 167.61 kilobytes arrives quickly even on a modest connection, and code splitting means the admin interface, which no teacher will ever open, never gets sent to their machine. The total static output of about seventeen megabytes is mostly deck artwork, which comes from a content delivery network and is only requested for the pack someone actually opens.

Figures 18 and 19 show two of these measurements as they were produced: the Lighthouse audit run against the live site, and the full test suite running.

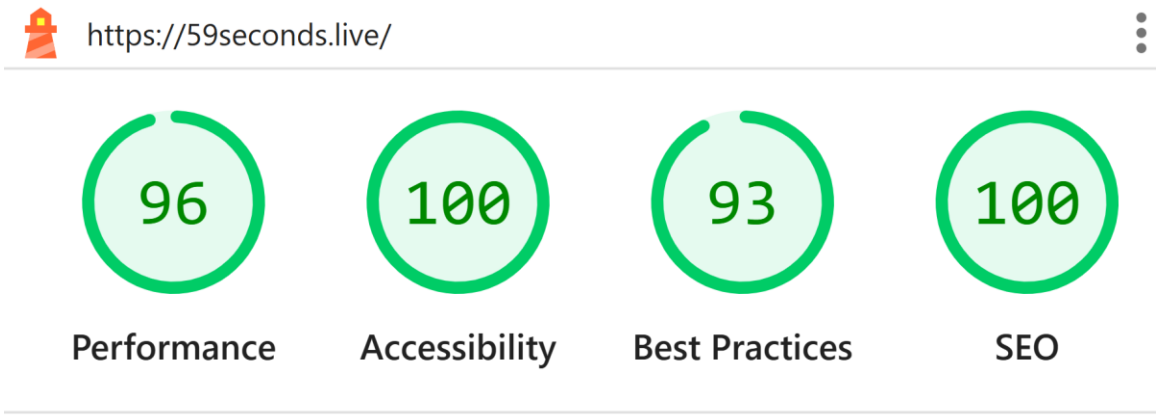

Figure 18. Lighthouse desktop audit of the live site.

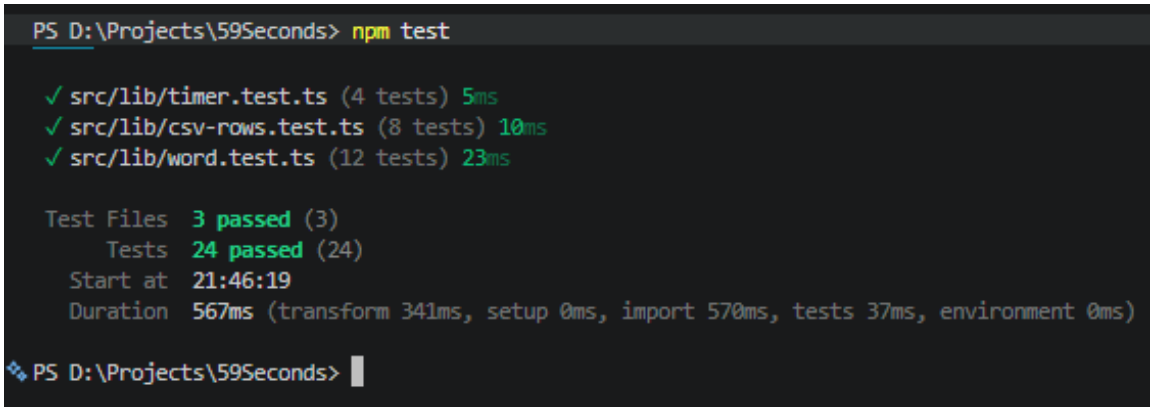

Figure 19. Vitest run at the end of Unit 6: 24 tests across three files, passing in 1.31 seconds.

Qualitative results

For the qualitative side I put the game in front of real classes. I used it with my own university-level students and with elementary students aged nine to twelve at the British Council, and a colleague tested it separately with younger learners. Six real changes came out of those sessions, all listed in Table 8. What they have in common is that no automated tool would have flagged a single one of them as a problem.

Observation	**Change made**
Children could not read the words and hints from their seats	Word and hint type on the play screen was substantially enlarged
Adding team scoreboards squeezed the card and shrank the type again	Navigation arrows were rebuilt as full-height side columns, returning width to the card
A colleague's young learners found all eleven packs too difficult	A twelfth Kids pack for ages six to seven was created; on retest the class responded well
A class wanted to name their own teams rather than play as Team 1 and Team 2	Team names were made editable directly on the play screen
The same cards reappeared before a round had finished	Shuffle logic was changed to exhaust the whole deck before repeating any card; six new unit tests were written to lock the behavior in
Children were too absorbed to notice the timer reaching zero	An audible bell was added at zero

Table 8. Changes arising from classroom testing.

Combined results

Table 9 pulls the evaluation together, and Figure 20 shows the quantitative measures against their recommended thresholds.

Metric	**Type**	**Result**	**Assessment**
Lighthouse Performance	Quantitative	96 / 100	Above threshold
Lighthouse Accessibility	Quantitative	100 / 100	No automated failures
Lighthouse Best Practices	Quantitative	93 / 100	Above threshold
Largest Contentful Paint	Quantitative	1.2 s	Target is 2.5 s
Cumulative Layout Shift	Quantitative	0.001	Effectively none
Warm query latency (median)	Quantitative	~65 ms	Cold start 1.10 s
Production build	Quantitative	55.3 s, 394 modules	Acceptable
Main bundle (gzipped)	Quantitative	167.61 kB	Admin code excluded
Unit tests	Quantitative	39 passing (24 at Unit 6)	Grew from 18 via fixes
Content scale	Quantitative	2,880 cards / 14,400 words	Counted, not estimated
Classroom usability	Qualitative	6 changes from live testing	Caught what metrics missed

Table 9. Consolidated evaluation results.

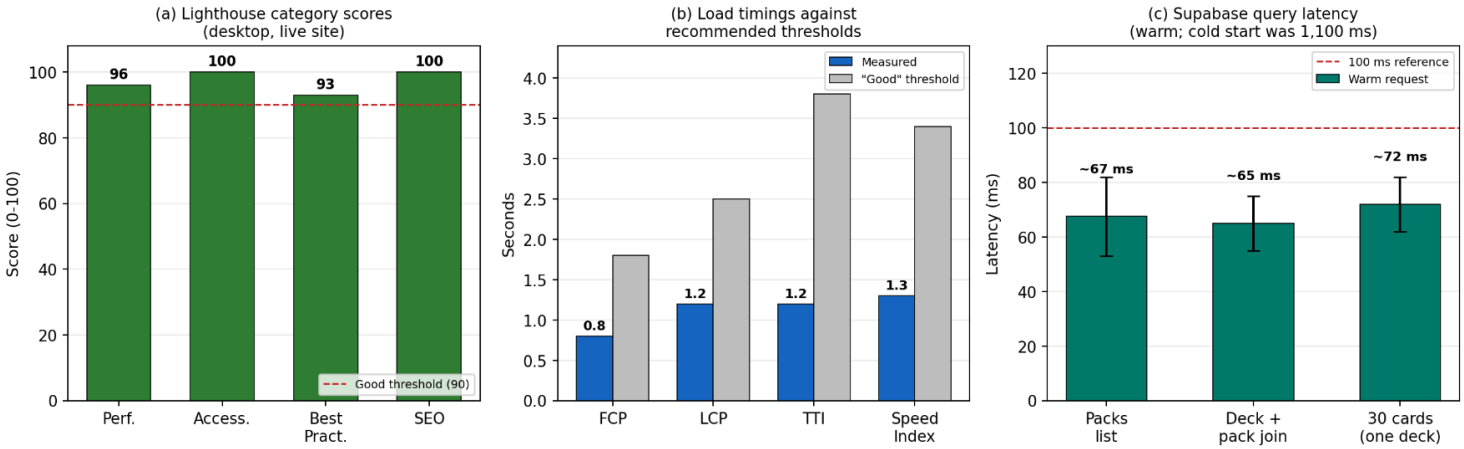

Figure 20. Lighthouse category scores, load timings against thresholds, and warm query latency.

4.4 System Testing and Defect Resolution

Unit 7 moved evaluation up a level, from testing modules in isolation to testing the integrated product against its requirements on the live production site. Fourteen test cases were designed across three levels, integration, system, and acceptance, and twelve were executed in real Chrome driven by Playwright, with screenshots captured at each step. The two not executed against production were the admin CSV upload, which writes real rows into the live database, and the full sign-in flow, which requires my personal credentials inside a script; both are documented as manual walkthroughs, and the sign-in gate and its redirect wiring were verified separately. Table 10 summarizes the results.

ID / Level	What it checks	Result
INT-02a / Integration	Sign-in gate and redirect wiring	Pass
INT-03 / Integration	Paywall on a locked deck	Pass
SYS-01 / System	Full round: shuffle, timer, bell, score, next card	Fail (bell), fixed, now Pass
SYS-02, SYS-03, SYS-04 / System	Anonymous play; deep link (SSR); fullscreen	Pass
ACC-01a-c / Acceptance	Desktop, tablet and phone viewports	Pass (all three)
ACC-02a-b / Acceptance	Self-explanatory controls; unprompted onboarding	Pass

Table 10. System test summary (Unit 7).

Eleven of the twelve executed checks passed on the first run. The failure was real: the bell that should ring when the timer reaches zero never sounded on the live site, confirmed across four different round durations in which the final-seconds countdown ticks played but the bell did not. The root cause was a race between three individually correct pieces: the bell fired inside the interval updater only when the remaining time fell

below a small threshold, but whole-number countdowns land on exactly zero through a different branch, and a second effect stops the timer at zero and tears the interval down before the bell line can ever run. What makes this a textbook regression is that the commit immediately before testing was the one that added the bell, and all 24 unit tests still passed, because none of them exercised the boundary where the countdown, the stop effect, and the sound meet.

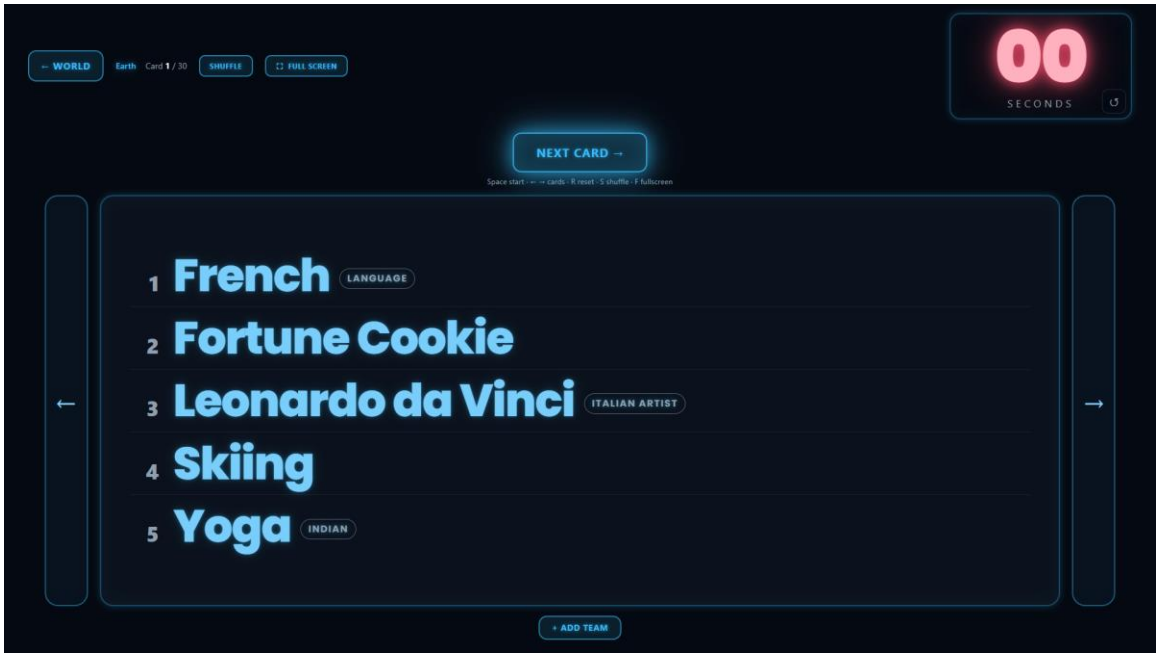

Figure 21. SYS-01: the timer at zero. The round completed but no bell sounded.

The fix extracted the logic into two pure functions, one for the countdown step and one for the bell decision, with the bell fired from its own effect on the same render where the timer lands on zero, so the teardown can no longer swallow it. A regression test now drives a complete round at six durations and asserts exactly one ring; running the old logic through the same simulation rings zero times, so the test genuinely guards the bug rather than describing the new code. After deployment, the production re-test rang four rounds out of four, the full system suite passed eleven of eleven, and the unit suite grew from 24 tests to 39. Two smaller findings remain documented rather than fixed, the client-and-server disagreement over CSV rows with a missing word, and case-sensitive deck-slug matching, both left alone deliberately because changing them touches data on a site taking real payments.

Production results after deploy:

Check	Before	After
Bell rings at zero (3s / 5s / 12s / 59s)	0/4	**4/4**
Bell strikes per round	0	1 (5 sine partials)
Countdown ticks	working	still working
Full system suite	10/11	**11/11**

Figure 22. Production verification after the fix: bell rings 4/4, full suite 11/11.

4.5 Summary of Evaluation Outcomes

Pulling the whole evaluation together, the system performs well on every number I measured and meets the requirements I set for it in Unit 2. Speed and stability are comfortably inside the recommended thresholds, the database answers in well under a tenth of a second once warm, the content catalogue is four times larger than the six packs I originally planned, and payments work end to end including the refund path that takes access away again.

The testing picture is more interesting than the metrics, because the two halves disagreed in a useful way. Thirty-nine unit tests pass in about a second and cover the logic I judged most likely to break. Twelve executed system tests cover the flows a teacher actually performs. Between them they found two production defects, and neither was found by the layer that looked most likely to catch it: the duplicate-card bug was found by a colleague teaching a lesson, and the silent bell was found by the first proper system test pass. In both cases the unit suite was green at the time. That is not an argument against unit testing, which is what makes the fixes safe to keep, but it is a clear argument that a suite passing is evidence about the pieces, not proof about the product.

DISCUSSION

5.1 Interpretation of Results

Four things stand out from the results, and only the first one is what I expected.

The one I did expect is that choosing managed infrastructure was the right call. I never had to write a login system, set up a database server, or build a deployment pipeline by hand. Almost all of my time went into the actual product. That is the only reason I could take this from a local prototype to a live product taking real money, on my own, inside the capstone window. The performance numbers back this up as well. A Largest Contentful Paint of 1.2 seconds and a median database response of about 65 milliseconds are not numbers I worked hard for. They are what the platform gives you by default.

The second is that treating content as data instead of code changed the economics of the whole project, and I did not see that coming when I designed it. When a co-worker told me the existing packs were too hard for six and seven year olds, adding a whole new pack, eight decks and 240 cards, was a content job rather than a coding job. I built the CSV import tool because I did not want to edit code every time I added words. It turned out to be the thing that let the product actually respond to its users.

The third is about the free deck in every pack. Anyone can play it with no account and no sign-up screen, so a teacher can go from a shared link to a running game in a couple of seconds. I originally did this for commercial reasons, so people could try before buying. In practice the effect has been more about teaching than selling, because the activity is easy enough to start that it gets used as a filler when a lesson finishes early, which is one of the most common ways it actually gets used. The paid bundle unlocks the other seven decks, and since access is a row in the database written only by a verified webhook, the free and paid paths run through exactly the same code.

The fourth finding is the one I did not expect, and it matters most. There is a big gap between what the automated tools measured and what real classrooms showed me. Lighthouse gave the site a perfect accessibility score of 100, and I want to be careful about what that actually means: it means no automated check failed. Color contrast passed, images had alt text, headings were in order, buttons had proper names. Then I put

the game in front of nine year olds and the first thing that happened was that they could not read the words, because the text was too small for how far away they were sitting. That is an accessibility failure by any sensible definition, and no tool caught it, because whether a child can read something from the back of a classroom is not what a scoring tool checks.

Something similar happened with bugs. The duplicate card problem, where the same cards kept appearing before the deck was finished, made it into production and was found by a co-worker during a lesson rather than by my tests. In software quality terms this is a defect escape, and defect escape rate is exactly the kind of measure that shows where testing does not match how people really use a system (Wells, 2025). I fixed the shuffle and wrote six new tests so it cannot come back, which is why my suite grew from 18 tests to 24, and the bell regression found in Unit 7 pushed it again to 39. The test count is therefore not just a number. It is a record of what testing and real users taught me.

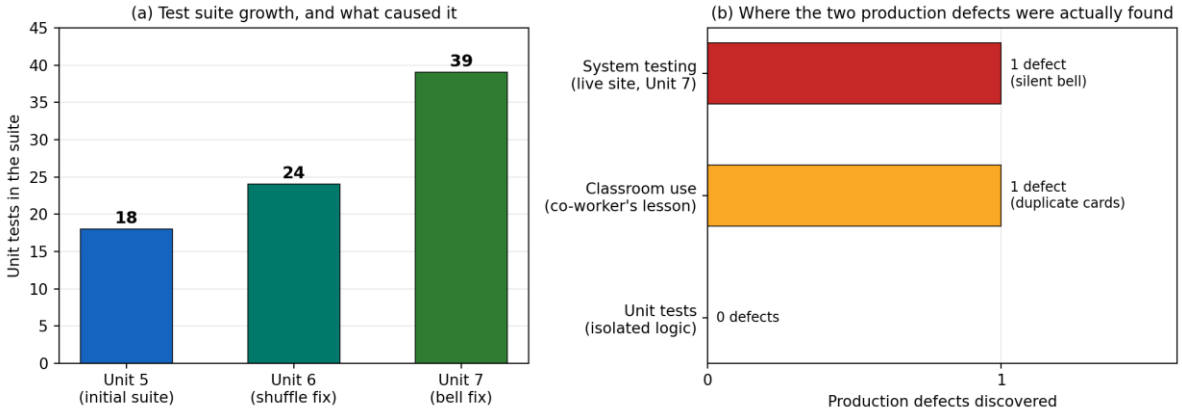

Figure 23. How the test suite grew, and where the two production defects were actually found.

Figure 23 puts the two findings side by side. The left panel shows the suite growing from 18 tests to 24 to 39, and it is worth saying plainly that neither jump came from me sitting down to write more tests. Each one came from a defect reaching real users, being fixed, and being locked in so it could not return. The right panel is the uncomfortable one: across the whole project my unit tests, run hundreds of times, found zero of the two defects that actually reached production. One was found by a colleague mid-lesson and the other by the first system test pass. The unit tests were not useless, they are why I could change the shuffle and the timer without breaking anything else, but they were measuring whether my code did what I already believed it did.

5.2 Comparison with Literature Findings

The results line up with the reading in several places, and in two of them I think my project adds a small piece of evidence rather than just borrowing an idea.

The clearest one is about reading text off a projector. Wang and Tahir (2020) reviewed 93 studies of Kahoot! and listed the complaints students make about game platforms shown on a classroom screen. Two of the most common were that the projected text was hard to read and that the time pressure felt stressful and inflexible. I used that as a design brief back in Unit 1, which is why 59 Seconds uses huge high contrast text, an editable timer, and never advances a card on its own. What surprised me is that my own classroom testing produced the same complaint anyway, even after I had designed for it. That suggests the readability problem is harder than it looks and that designing for it once is not enough. The research predicted the problem, and my classroom confirmed it and then showed me how much further I still had to go.

The second is about how to evaluate software. LaunchNotes (2023) argues that quantitative and qualitative metrics balance each other and that using both gives a fuller picture than either on its own. My project is a fairly blunt example. If you only looked at my numbers, you would say the app is fast, stable and accessible. If you only looked at the classroom feedback, you would find six real usability problems but learn nothing about performance. You need both to describe the system honestly, and in my case it was the qualitative half that found the problems that mattered most to the people using it.

On infrastructure, what happened here supports the argument from Jonas et al. (2019) that serverless and managed platforms simplify cloud development in the same way high level languages simplified programming, by taking system administration away from the developer. The evidence is indirect but it fits: one person built the whole thing, with security features that would normally take deliberate effort, because the platform supplied them as defaults.

On configuration, Summers (2020) describes software configuration management as the discipline that keeps a project's parts consistent and protects the integrity of the product. Two incidents show what it costs when that goes wrong. The first was setting a production environment variable by piping a value in from PowerShell, which quietly added a byte order mark to the front of it and corrupted the webhook signing secret by

one invisible character. Every webhook after that failed its signature check, and because the platform hides sensitive values I could not simply look at it. I worked it out in the end by comparing the length of the stored secret against what it should have been. The second was simpler but just as frustrating: a fix I pushed only to the development branch did not appear on the live site, twice in one day, because only main deploys to production. Neither was a coding mistake. Both were configuration problems, and both were solved by writing the rule down.

It is worth checking the project against a fuller description of the discipline. Bennett (2024) splits software configuration management into four parts: version control, build management, change tracking, and release automation. On the first three I am in decent shape, since code and database migrations live in Git, builds are produced by the platform rather than by hand, and milestones are tagged with proper release notes. On the fourth I am only partly there. Releases are automatic in the sense that merging to main builds and deploys with no input from me, but nothing checks the change on the way through. That missing check is the same gap I identified in Unit 4 and still have not closed, and putting it inside Bennett's four parts made it clearer that this is not a small thing I forgot but a whole missing piece of an otherwise complete setup.

My testing approach came fairly directly from the reading too. Amazon Web Services (n.d.) describes unit testing as checking individual pieces of code in isolation, and the point of my Unit 5 refactor was to make that isolation possible by pulling logic out of React components and server functions into small standalone files I could import and test on their own. Rana (2023) sets out what a good test case looks like in terms of preconditions, inputs and expected results, and all 24 of my tests follow that shape. The catch is that testing pieces in isolation tells you nothing about how they behave together, and both of the bugs that reached production lived in exactly that gap. Unit 7 closed part of it: system testing evaluates the fully integrated product against its requirements rather than its pieces (Hrupa, 2024; BrowserStack, 2025), and the first full pass earned its keep immediately by finding the one defect that all the isolated tests had missed.

Keeping my documentation current follows what Brown (2025) recommends, which is that documentation should be organized, easy to reach, and actually maintained instead of written once and abandoned. I keep a single PROJECT_CONTEXT.md file in the repository holding the architecture, the conventions, and every mistake that has cost me time, and I update it as part of the work rather than afterwards.

Finally, my deployment setup is a simple version of the staged approaches described in the deployment reading (Sumo Logic, n.d.). Pushing to development gives me a preview deployment that runs the real build on real infrastructure without touching live users, and only merging to main puts a change in front of anyone. This is not blue-green or canary deployment properly, since I am not splitting traffic between two versions, but it gives me the main benefit, which is testing a real build in a realistic environment first. Rolling back means promoting an older deployment instead of rebuilding anything.

5.3 Implications and Limitations

The practical takeaway is that the barrier to building useful classroom software has dropped a long way. One teacher who can code is now able to design, build, secure, deploy and sell a tool that runs in real lessons, on infrastructure that costs very little at this size. The wider point for educational technology is that the one screen, many students model deserves more attention than it gets, especially for speaking practice, where the whole idea is students looking at each other instead of at their own devices.

The limitations are real and I would rather state them plainly. The biggest is that the pedagogical claim behind this project, that a game like this lower speaking anxiety and gets students talking more, has not been tested properly. The classroom feedback I gathered is genuine but informal and unstructured, and it comes from my own students and one co-worker's classes, which is a convenience sample. There was no control group, I did not measure how much students actually spoke, and I did not use any anxiety scale. I am also both the developer and one of the teachers, which is an obvious source of bias.

The technical limitations are narrower. My automated unit tests only cover pure logic. Unit 7 added a documented system test suite, driven by Playwright against the live site, but it runs on demand rather than in a pipeline, and the payment and admin upload flows are still verified through manual walkthroughs rather than automation. The tests are also not enforced, since there is still no pipeline running them automatically, so nothing stops an untested commit reaching production except me remembering, which is exactly the gap I wrote about in Unit 4. Two known inconsistencies from Unit 5 are also still there on purpose, because fixing them means touching data on a site that now takes real payments: the client side importer counts a row as valid if it has a deck name but a missing word, which the server then rejects, and deck matching is case sensitive, so a differently capitalized slug quietly creates a duplicate deck instead of matching the existing one. The repository also carries about 1,300 old line ending lint errors and four unrelated type errors.

There is another set of limitations that is not technical at all. I am the only person maintaining this, so if I am unavailable nobody can deploy a fix, rotate a leaked key, or answer a refund request. The project also leans on single providers for payments, database, login and hosting, which means a price change or a policy decision at any one of them lands directly on a live product with paying customers. The content has a similar weakness. All twelve packs were put together by one teacher, and while I can speak with some confidence about what a Korean or South African classroom will recognize, several of the regions I have built packs for are places I have never taught in. There is a real chance some of those word choices feel right to me and slightly off to the students they are meant for.

On the evaluation itself I want to be careful about accessibility. Scoring 100 in Lighthouse means no automated check failed. It does not mean the app has been tested with screen readers, with assistive technology, or with visually impaired students, because none of that has happened. Performance was also measured from one location on a desktop connection, with no load testing for what would happen if a whole school used it at once, and the 1.10 second cold start is still the slowest thing in the system.

5.4 Future Work

The most valuable technical improvement is also the one I have put off longest: a workflow that automatically runs the linter, a production build and the full test suite on every pull request into main. I already have the tests and I already have automatic deployment. What is missing is the gate between them. Adding it would finish the pipeline I described in Unit 4 and make my test suite mean something day to day, instead of only when I remember to run it.

A few smaller fixes are already well defined. The four Performance points I lost in Lighthouse come almost entirely from the biggest image on the page being lazy loaded, which is a small and easily measured fix. The two CSV inconsistencies should be sorted so the client and the server agree on what a valid row is, and slug matching should become case insensitive with a migration to merge any duplicates that already exist. The old line ending errors deserve one cleanup commit.

The bigger future work is not technical. The obvious next step is a proper classroom study measuring what I have been claiming: how much students actually speak, how evenly participation is spread across a class, and a real anxiety measure taken before and after a period of use, with something to compare against. That would turn the main idea behind this project from a reasonable expectation into an actual result. A genuine accessibility audit using assistive technology, and load testing for whole school use, would do the same for the other two claims I cannot currently back up.

After that, the direction my users have pointed me in is clear. The Kids pack showed that splitting content by age meets a real need, which argues for more age levels rather than simply more countries. Teacher accounts with saved settings, and letting teachers build their own packs, are natural extensions of the import pipeline I already have. One feature stays off the list on purpose, which is letting students use their own phones as buzzers. It would not be hard to build, but it brings back exactly the heads down, everyone on their own screen problem this project exists to avoid, and I would want evidence that it is worth it before adding it.

CONCLUSION AND FUTURE WORK

I started this capstone with a problem I run into constantly as an English teacher. My students can speak English, but often they will not, and the technology schools bring in to help usually makes things worse by putting every learner behind their own screen with content that means nothing to them. 59 Seconds was my answer: a projector first speaking game with vocabulary built around the region the students actually live in, running on hardware classrooms already own.

The project delivered more than it promised. The system is finished and live at 59seconds.live, with twelve region packs holding 2,880 cards and 14,400 words, and it takes real payments, which I have tested end to end including a refund that correctly removed access again. It performs well on every measure I took: a Lighthouse Performance score of 96, a Largest Contentful Paint of 1.2 seconds, almost no layout shift, and a median warm database response of about 65 milliseconds. It has 39 automated unit tests plus a documented system test suite, its milestones are recorded as tagged GitHub releases, and its setup and deployment steps are written down. Going live, switching payments from sandbox to production, and adding a twelfth pack were all outside the scope I originally set myself.

What this project contributes is a demonstration rather than a discovery. It shows that one developer, working alone on managed infrastructure over a few weeks, can build and ship a secure, paid, classroom ready application without giving up the things that usually need a team: access control enforced at the database, payment webhooks verified with cryptography, database changes tracked in version control, and automated tests. It also adds a small piece of evidence for a design position that runs against where most educational technology is heading, which is that for speaking practice specifically, one shared screen with students facing each other may simply work better than a device each.

The most useful thing I learned came out of the evaluation rather than the build. Quantitative and qualitative metrics are not the same thing measured at different levels of detail. They measure different things entirely. An app that scored a perfect 100 for accessibility was, on the same day, unreadable to the nine year olds I built it for. Every one of the six most important improvements I made this unit came from watching real

classes rather than from a tool, and one of them was a bug that had slipped past a test suite written specifically to cover that part of the code; Unit 7 then found a second, the silent end-of-round bell, the same way. Automated measurement tells you whether your system is correct according to your own definitions. Only users tell you whether those definitions were right in the first place.

Two other lessons are worth recording. Configuration mistakes cost far more than they should because you cannot see them: one invisible character, which nothing logged and nothing displayed, broke the most security critical part of my payment system and took hours to track down. And decisions that look like conveniences can turn out to be structural. I built the CSV importer to save myself from editing code, and it is the reason I could answer a co-worker's feedback about young learners by shipping a whole new pack instead of shrugging.

Most of my challenges were of that kind: character encoding corruption, webhook secrets, secrets I had committed early in the repository's history, and confusion over which branch deploys where. Each one was fixed and then written down, so the fix outlives the incident. The clearest next step is still the automated check that would run my existing tests before anything reaches production, followed by a properly designed classroom study to test the idea that started all of this. As it stands, 59 Seconds is a complete run through a full development lifecycle, from a teacher being frustrated in a classroom, through research, design, building, testing and evaluation, to a live product that real students are playing.

6.1 Summary of Contributions

It is worth being precise about what this project contributes, because "I built an app" is not a contribution. There are three.

The first is a working demonstration that the barrier to building real classroom software has dropped a long way. One teacher who can code, using managed infrastructure over about eight weeks, produced a secure, paid, deployed product with database-enforced access control, cryptographically verified payment webhooks, versioned database migrations and an automated test suite. Ten years ago most of that list would have needed a team and a budget.

The second is a piece of evidence for a design position that runs against where most education technology is going. The industry assumption is one device per student. For speaking practice specifically, my classroom experience suggests the opposite arrangement is better: one shared screen, students facing each other, because the whole point is the talking between them rather than the interaction with a device. I cannot prove that claim with the evidence I have, and Chapter 5 says so, but the product exists and is used, which is a starting point.

The third is the content model. Treating vocabulary as data in a CSV pipeline rather than as something written into the code turned out to matter more than I expected. It is the reason a request from one co-worker about young learners could be answered with an entire new pack in a day rather than a sprint, and it is the part of this project I would carry into anything I build next.

6.2 Lessons Learned

The lesson I keep coming back to is that automated measurement tells you whether a system is correct by its own definitions, and only users tell you whether those definitions were the right ones. A perfect accessibility score and unreadable text coexisted in the same build on the same day. Every one of the six most valuable changes I made came from watching a real class rather than from a tool.

Two smaller lessons are worth writing down. Configuration mistakes cost far more than their size because they are invisible: one byte-order-mark character, which nothing logged and nothing displayed, broke the most security-critical path in the payment system and took hours to find. And decisions that look like conveniences at the time can turn out to be structural, which is exactly what happened with the CSV importer.

6.3 Future Work

The immediate technical priority has not changed since Unit 4: a continuous integration workflow that runs linting, a production build and the full test suite automatically on every pull request into main. I have the tests and I have automatic deployment; the gate between them is the missing piece. After that come the specific fixes named in Chapter 5, the lazy-loaded hero image costing four Lighthouse points, the

client and server disagreement over CSV rows, and case-insensitive deck matching with a migration to merge any duplicates.

The more valuable future work is not technical at all. The central claim behind this whole project, that a game like this lowers speaking anxiety and increases how much students actually talk, is still untested in any controlled way. A proper classroom study measuring student talk time, how evenly participation spreads across a class, and a validated anxiety measure taken before and after a period of use would turn that claim into a result. A genuine accessibility audit with assistive technology and load testing for whole-school use would do the same for the other two claims I currently cannot back up.

REFERENCES

Amazon Web Services. (n.d.). What is unit testing? AWS. https://aws.amazon.com/what-is/unit-testing/

Bennett, L. (2024, August 13). Software configuration management in software engineering. Guru99. https://www.guru99.com/software-configuration-management-tutorial.html

BrowserStack. (2025, May 29). What is system testing? (Examples, use cases, types). BrowserStack. https://www.browserstack.com/guide/what-is-system-testing

Brown, J. (2025, December 2). Knowledge software documentation best practices [with examples]. Helpjuice. https://helpjuice.com/blog/software-documentation

Hrupa, A. (2024, March 19). What is system testing in software testing. Testfort. https://testfort.com/blog/what-is-system-testing-in-software-testing

Jonas, E., Schleier-Smith, J., Sreekanti, V., Tsai, C.-C., Khandelwal, A., Pu, Q., Shankar, V., Carreira, J., Krauth, K., Yadwadkar, N., Gonzalez, J. E., Popa, R. A., Stoica, I., & Patterson, D. A. (2019). Cloud programming simplified: A Berkeley view on serverless computing (arXiv:1902.03383). arXiv. https://arxiv.org/abs/1902.03383

Krashen, S. D. (1982). Principles and practice in second language acquisition. Pergamon Press. http://www.sdkrashen.com/content/books/principles_and_practice.pdf

LaunchNotes. (2023, September 1). Qualitative vs quantitative metrics: A comprehensive comparison. LaunchNotes. https://www.launchnotes.com/blog/qualitative-vs-quantitative-metrics-a-comprehensive-comparison

Oguz, A. (2025). Project management (2nd ed.). MSL Academic Endeavors. https://pressbooks.ulib.csuohio.edu/projectmanagement2ndedition/

OWASP Foundation. (2021). OWASP Top 10:2021. https://owasp.org/Top10/

Paddle. (n.d.). Verify webhook signatures. Paddle Developer Documentation. https://developer.paddle.com/webhooks/signature-verification

Rana, K. (2023, September 23). What is a test case? Test case examples. ArtOfTesting. https://artoftesting.com/test-case

Red Hat. (2025, April 29). What is DevOps? Red Hat. https://www.redhat.com/en/topics/devops/what-is-devops

Summers, B. L. (2020). Effective methods for software engineering. Auerbach Publishers.

Sumo Logic. (n.d.). What is software deployment? Sumo Logic. https://www.sumologic.com/glossary/software-development/

Sung, Y.-T., Chang, K.-E., & Liu, T.-C. (2016). The effects of integrating mobile devices with teaching and learning on students' learning performance: A meta-analysis and research synthesis. Computers & Education, 94, 252-275. https://doi.org/10.1016/j.compedu.2015.11.008

Supabase. (n.d.). Row level security. Supabase Documentation. https://supabase.com/docs/guides/database/postgres/row-level-security

Vercel. (n.d.). Vercel documentation. https://vercel.com/docs

Wang, A. I., & Tahir, R. (2020). The effect of using Kahoot! for learning - A literature review. Computers & Education, 149, 103818. https://doi.org/10.1016/j.compedu.2020.103818

Wells, J. (2025, April 16). Top 12 software quality metrics to measure and why. LinearB. https://linearb.io/blog/software-quality-metrics

APPENDICES

Appendix A: Database Schema Summary

Table	Key columns	Purpose and access rules
packs	id (PK), slug, name, country, flag_emoji, accent_color_index, is_free, price_cents, position	One row per regional pack (11 total). Public read; service-role write.
decks	id (PK), pack_id (FK → packs, CASCADE), slug, name, display_name, accent_color_index, position, is_free	Eight decks per pack; deck at position 0 is the free preview. Public read; service-role write.
cards	id (PK), deck_id (FK → decks, CASCADE), position, word1…word5	The five words shown per game card. Public read; service-role write.
bundle_entitlements	user_id (FK → auth.users), pack_slug, Paddle transaction/customer IDs	One row per pack a user owns. Written only by the verified Paddle webhook.
user_roles	user_id (FK → auth.users), role	Grants /admin access when role = 'admin'.

Table A1. Database schema summary.

Appendix B: Admin CSV Import Format

Decks and cards are imported through /admin as UTF-8 CSV files. Each row supplies the deck slug in the Source.Name column and the card's five words in Column2 through Column6. The deck slug must follow the pattern <CountryToken>S01Deck<N> (for example, KoreaS01Deck3); rows sharing a slug are grouped into one deck, and a slug that does not yet exist creates a new deck automatically. The import is append-only: it never modifies existing decks or cards, so replacing a deck's content is an explicit delete in the deck editor followed by a fresh import. Files must be saved as "CSV UTF-8" - plain ANSI CSV irreversibly corrupts non-Latin characters such as Korean.

Source.Name	Column2	Column3	Column4	Column5	Column6
KoreaS01Deck1	김치 (kimchi)	한강 (Han River)	지하철 (subway)	노래방 (karaoke)	설날 (Lunar New Year)
KoreaS01Deck1	비빔밥 (bibimbap)	부산 (Busan)	태권도 (taekwondo)	한복 (hanbok)	치맥 (chicken & beer)

Table B1. Example rows from a UTF-8 deck CSV.

Appendix C: System Test Case Log

The full log of the system test pass carried out in Unit 7 against the live site, driven by Playwright in real Chrome, with a screenshot captured at each step.

ID	Level	What it checks	Result
INT-01	Integration	CSV upload through to cards visible in the game	Documented; not run on production (writes real rows)
INT-02	Integration	Full sign-in flow with credentials	Documented; not run (requires personal credentials)
INT-02a	Integration	Sign-in gate href and redirect wiring	Pass
INT-03	Integration	Paywall and preview limit on a locked deck	Pass
SYS-01	System	Full round: shuffle, timer, bell, scoring, next card	Fail (bell), root-caused, fixed, re-tested Pass
SYS-02	System	Anonymous free-deck play with no account	Pass
SYS-03	System	Deep link to a pack URL, server-rendered	Pass
SYS-04	System	Fullscreen mode	Pass
ACC-01a	Acceptance	Desktop viewport 1920x1080, no overflow	Pass
ACC-01b	Acceptance	Tablet viewport 820x1180, no overflow	Pass
ACC-01c	Acceptance	Phone viewport 390x844, no overflow	Pass
ACC-02a	Acceptance	Controls usable without instructions	Pass
ACC-02b	Acceptance	Onboarding shown unprompted to a new visitor	Pass

Table C1. Full system test case log (Unit 7).

Appendix D: Maintenance Schedule

The routine maintenance plan for the system after deployment, kept deliberately light because it is run by one person alongside a teaching job.

Frequency	Activity	Maintenance type
Every release	Full unit suite, typecheck, and the system checklist for anything touching the play screen or payments	Preventive
Weekly in term	Lighthouse run on the live site; scan Paddle dashboard and Supabase logs	Preventive
Monthly	Dependency updates on the development branch, verified by preview deployment	Adaptive
Quarterly	Rotate the service-role key and webhook secret; re-run the full system suite	Preventive
As reported	Bug fixes, logged as GitHub issues and locked in with a regression test	Corrective
As requested	Feature and content changes driven by classroom feedback	Perfective

Table D1. Post-deployment maintenance schedule.